Advanced Software Engineering VU [194.187]

Maria Christakis

TU Wien

https://mariachris.github.io

Summary

Abstract interpretation: Terminology

- Abstract domains are standard abstractions for building abstract interpreters
  - They abstract program data, basic operations, and control
- An abstract program state is a map from program variables to elements in the abstract domain
- An abstract transformer describes the effect of statement and expression evaluation on an abstract state

Example – Step 1: Select abstraction

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= x + y;
}
```

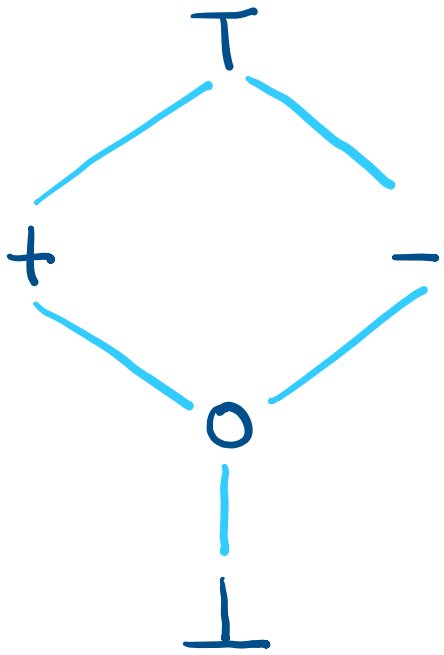
pc: 2

x : +

y : ⊥

i : ⊤

example abstract state in the sign domain

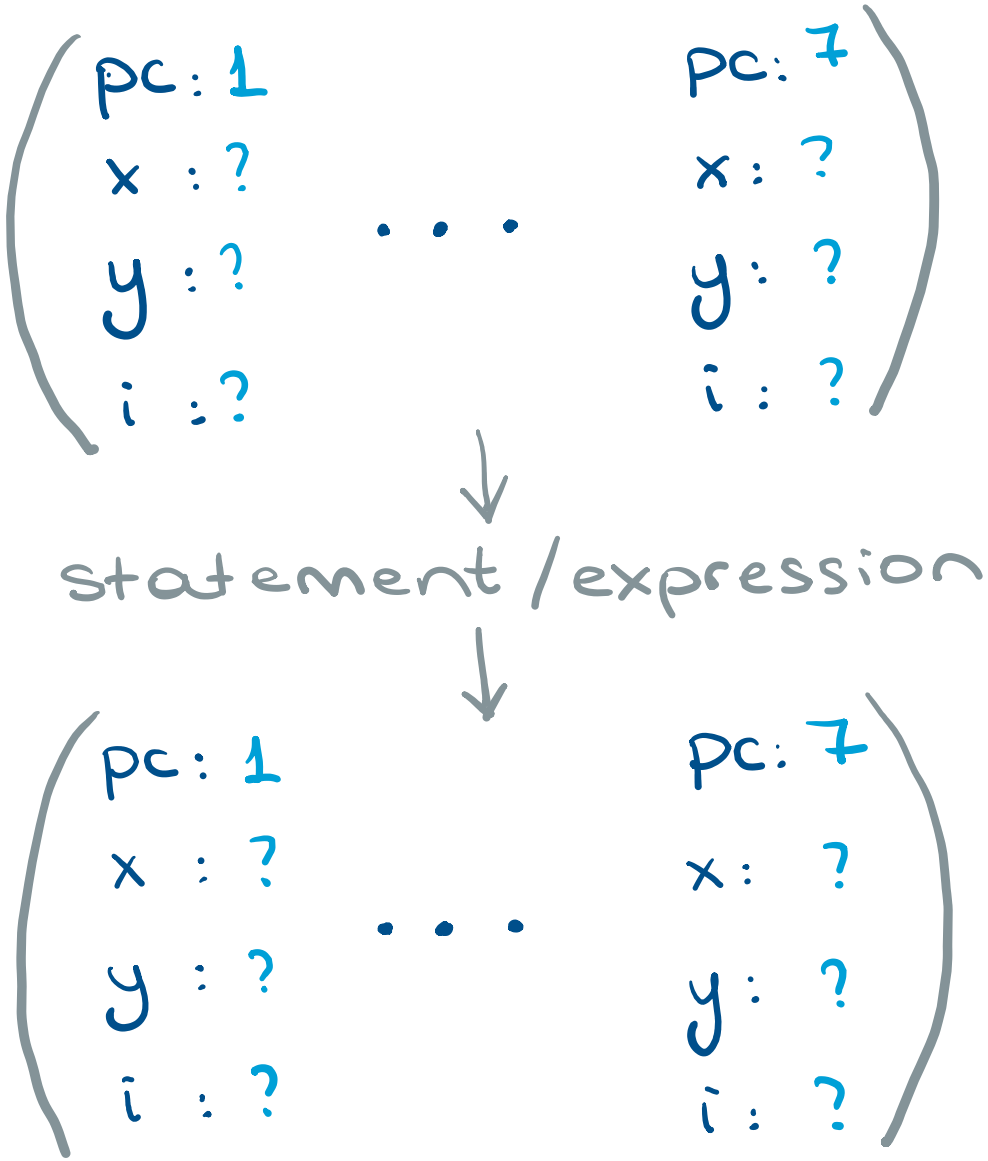


Example – Step 2: Define transformers

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= x + y;
}
```

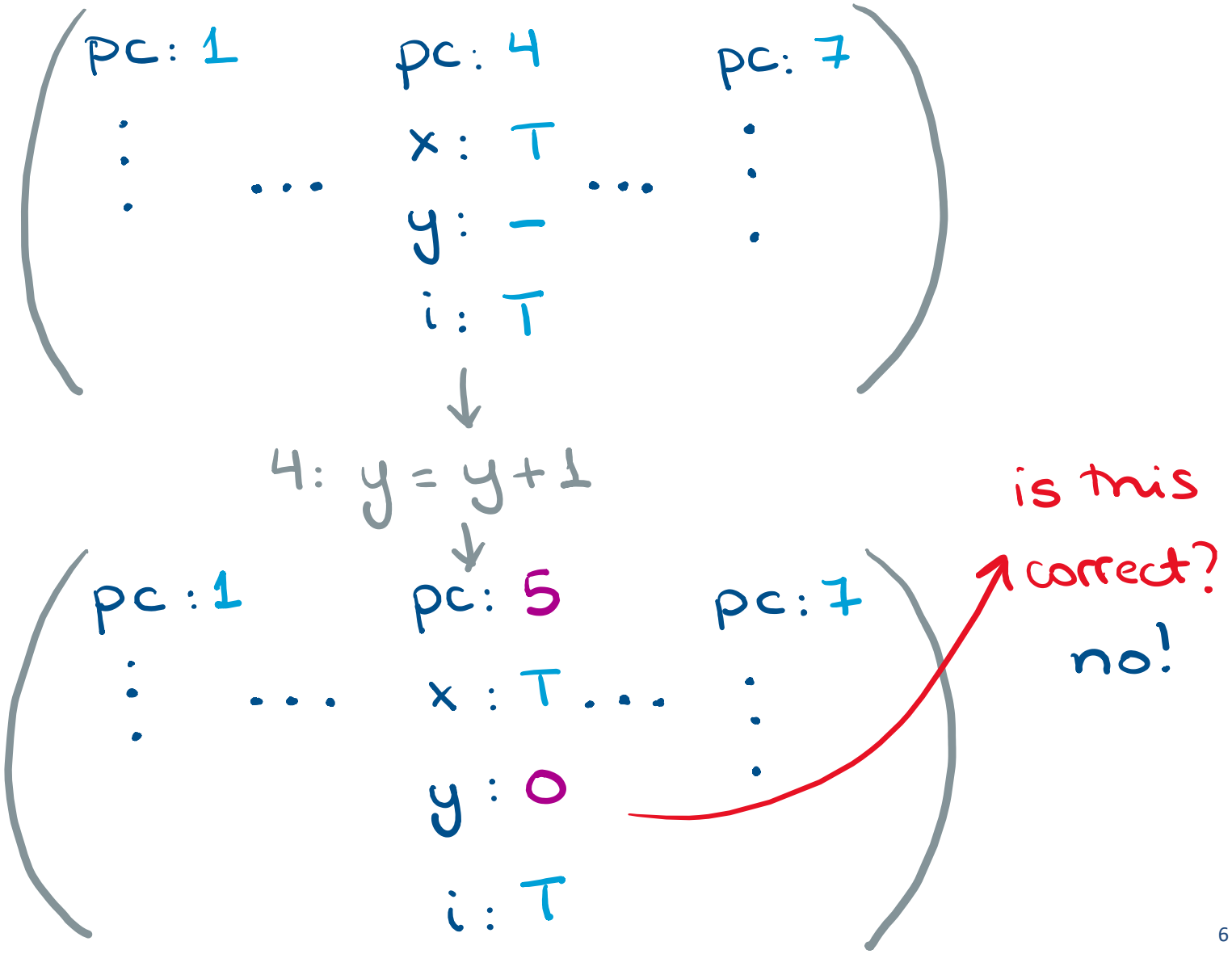


Example – Step 2: Define transformers

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= x + y;
}
```

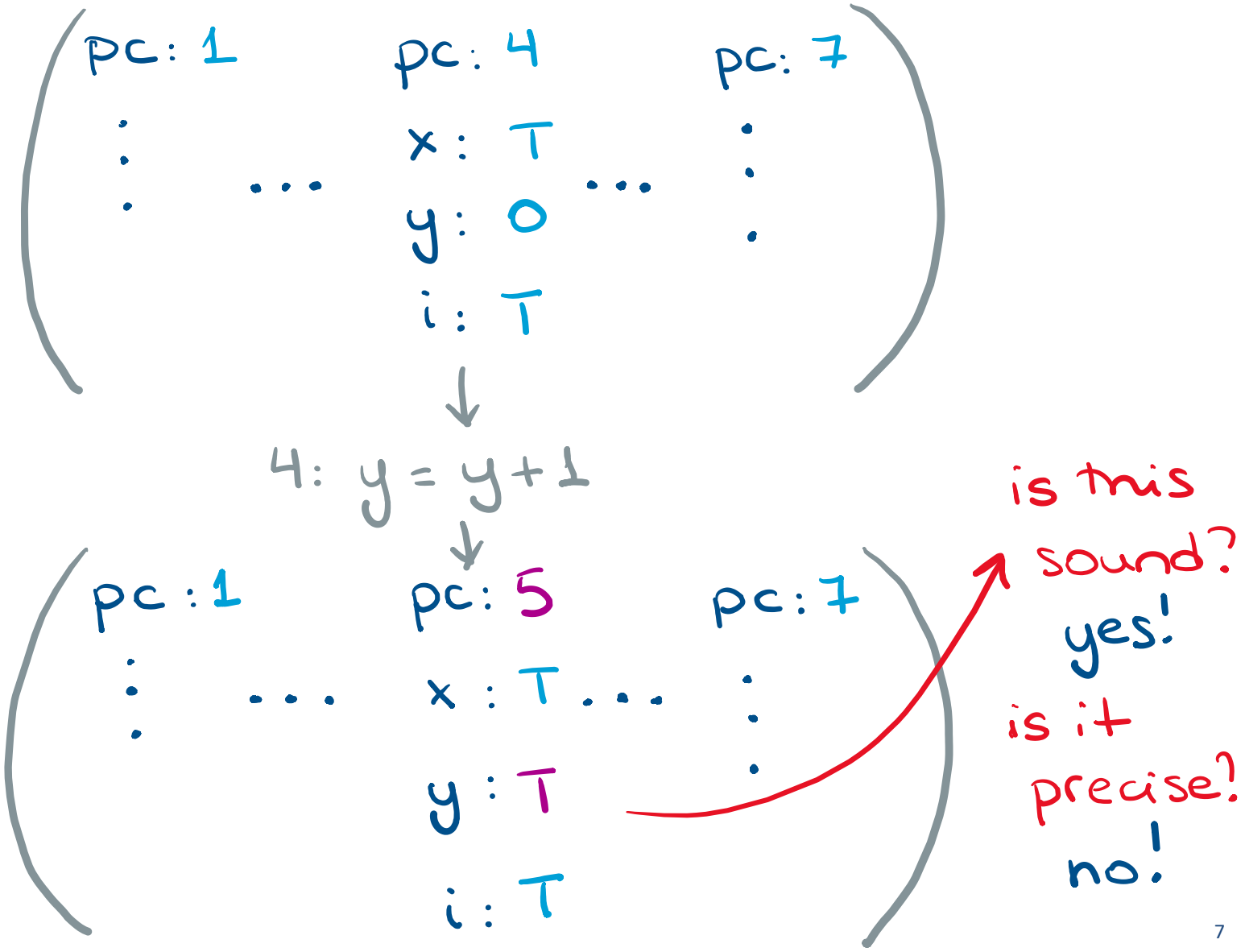


Example – Step 2: Define transformers

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= x + y;
}
```



Abstract transformers: Properties

- It is easy to be sound and imprecise
  - Simply output T
- It is desirable to be both sound and precise
- If we lose precision, it needs to be clear why and where
  - Sometimes, computing the most precise (and sound) transformer, called the best transformer, is impossible
  - For efficiency reasons, we may sacrifice some precision

Abstract interpretation: Terminology

Abstract interpretation involves three steps:

1. Selecting an abstract domain

2. Defining abstract transformers

3. Iterating the transformers over the domain to obtain an over-approximation
   - We iterate until we reach a fixed point, which is the over-approximation of the program

4. Check the properties

If the over-approximation is correct, then the program must be correct

Abstract interpretation: Theory

Example – Step 3: Iterate to fixed point

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= x + y;
}
```

Start with the least abstract element

pc: 1
x: ⊥
y: ⊥
i: ⊥

pc: 2
x: ⊥
y: ⊥
i: ⊥

pc: 3
x: ⊥
y: ⊥
i: ⊥

pc: 4
x: ⊥
y: ⊥
i: ⊥

pc: 5
x: ⊥
y: ⊥
i: ⊥

pc: 6
x: ⊥
y: ⊥
i: ⊥

pc: 7
x: ⊥
y: ⊥
i: ⊥

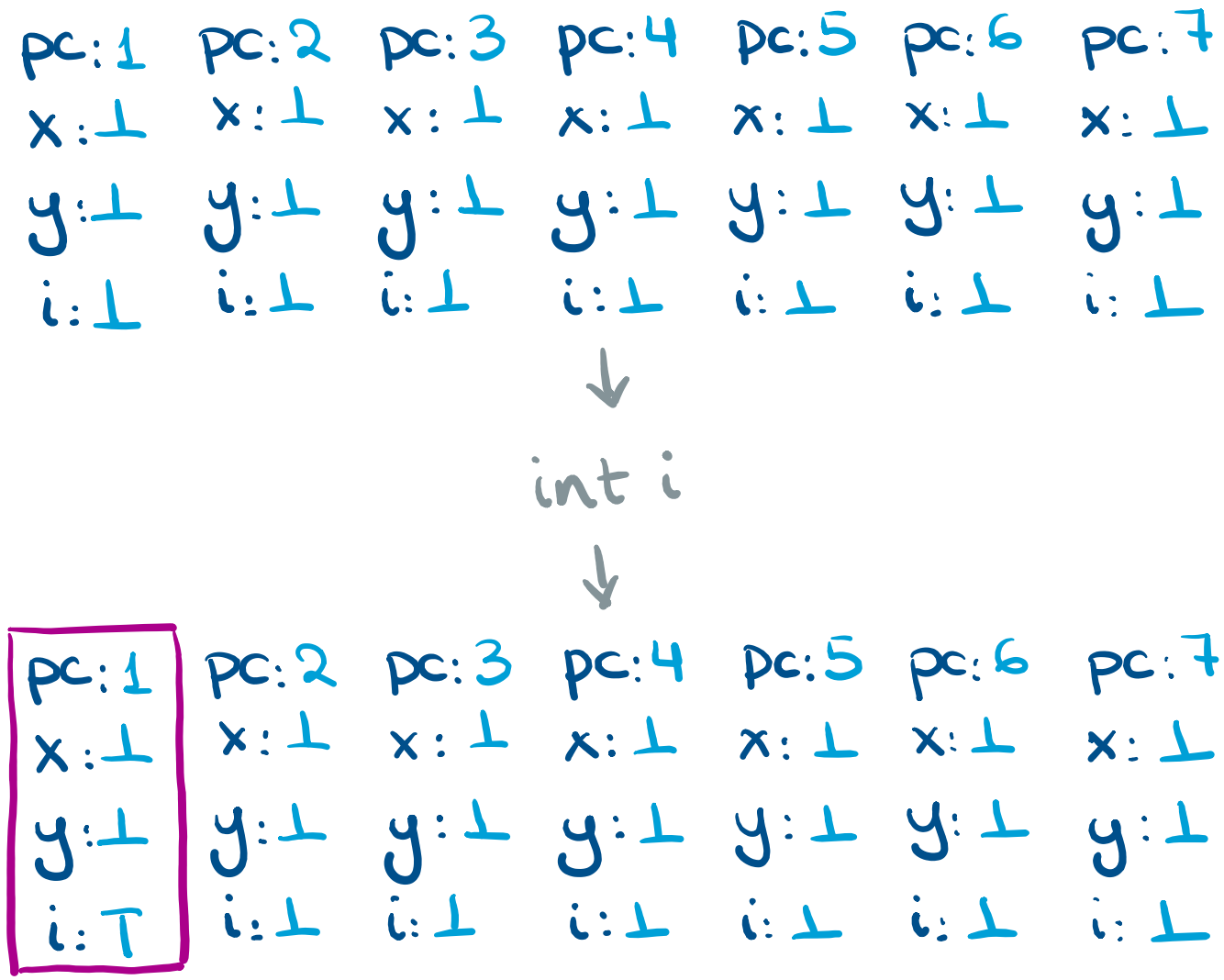
Let's do some iterations!

Example – Step 3: Iterate to fixed point

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= x + y;
}
```

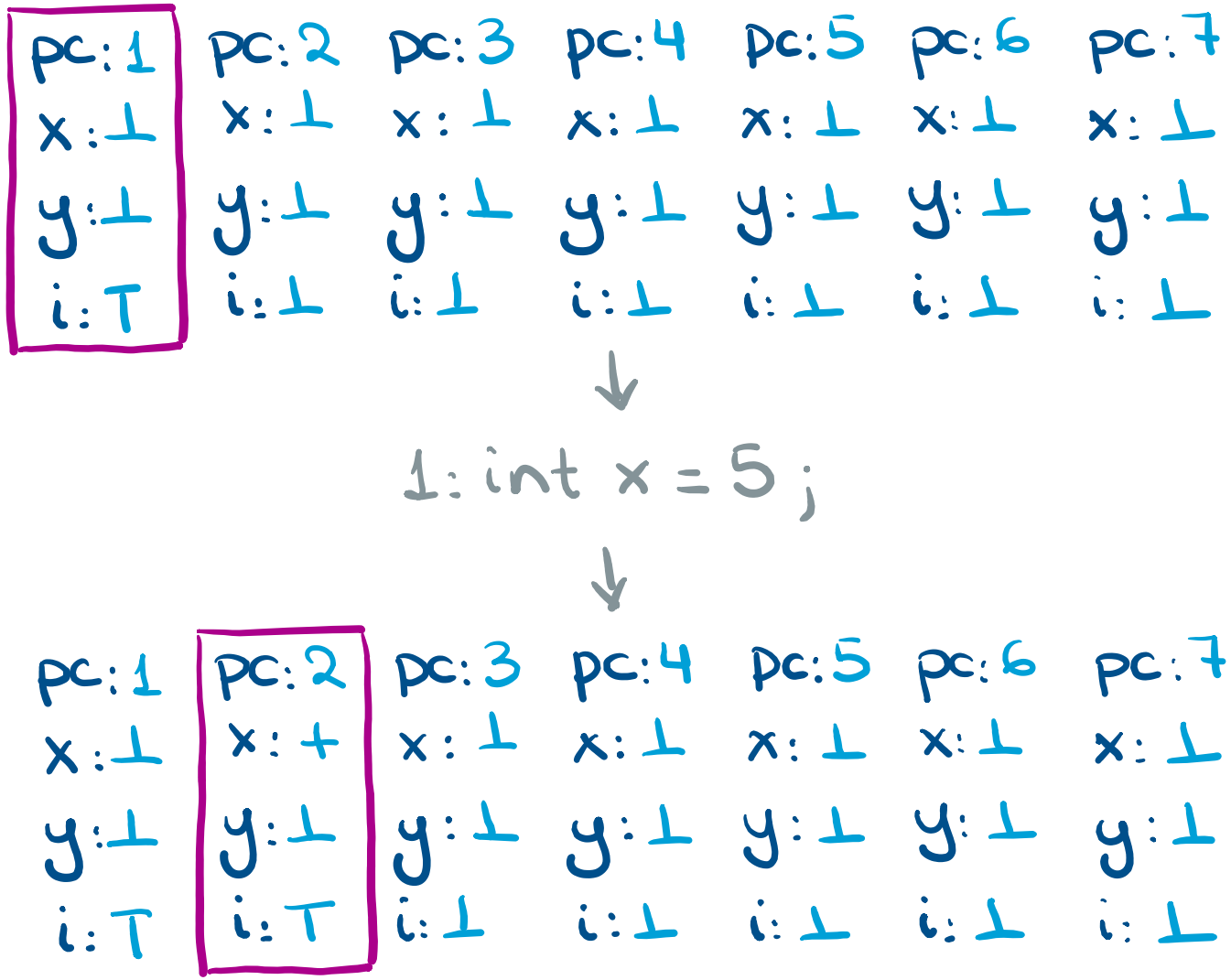


Example – Step 3: Iterate to fixed point

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= x + y;
}
```

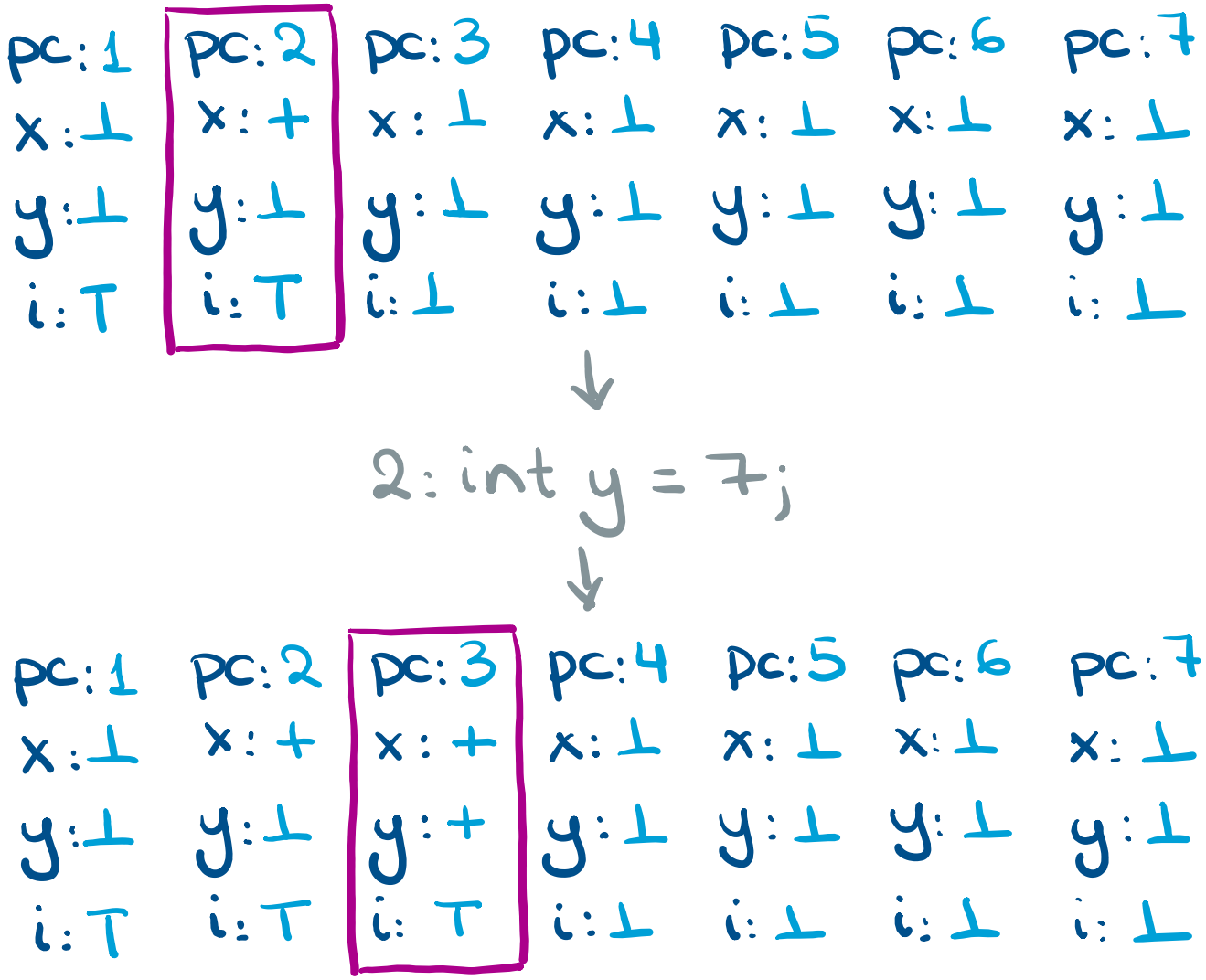


Example – Step 3: Iterate to fixed point

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= x + y;
}
```

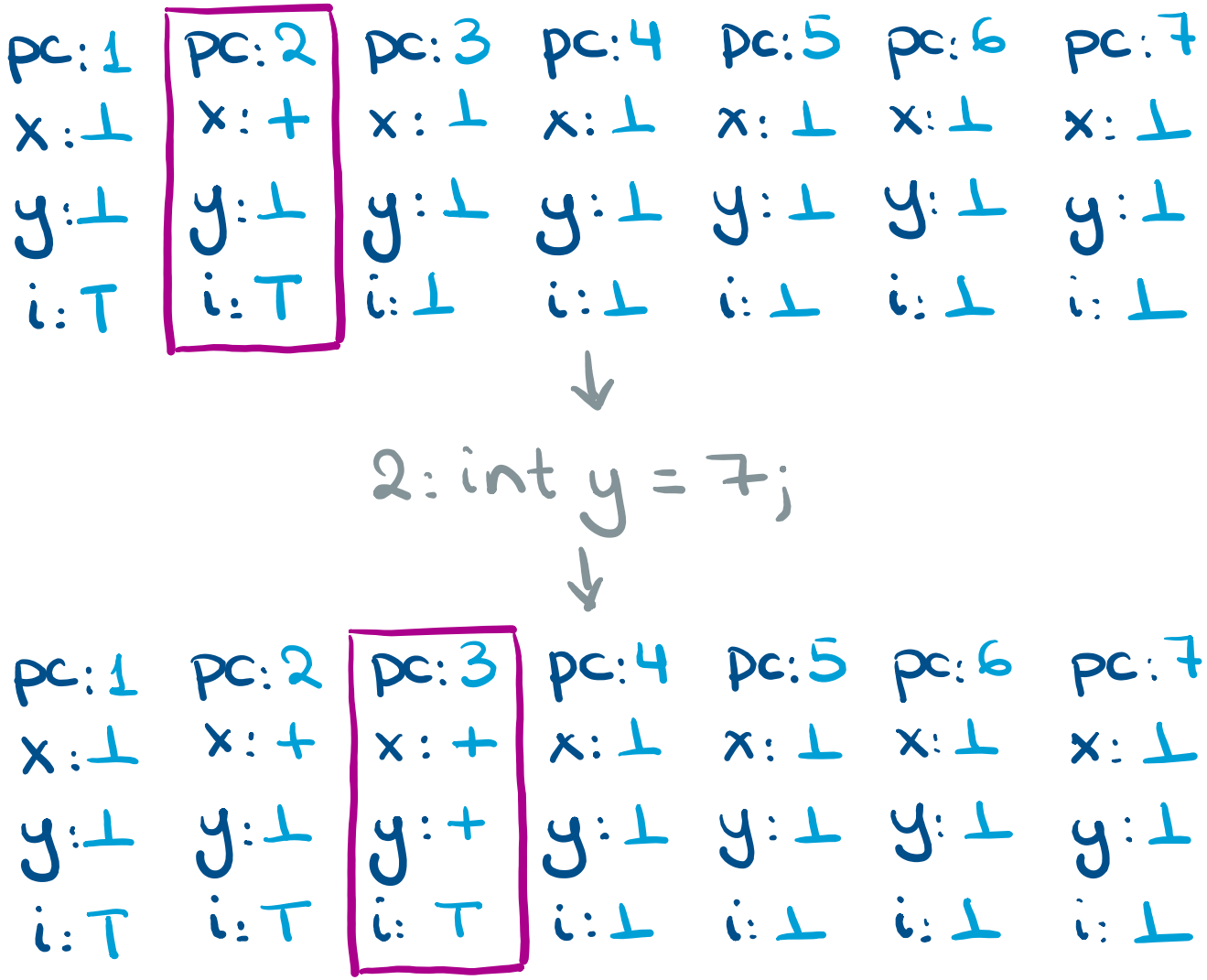


Example – Step 3: Iterate to fixed point

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= x + y;
}
```



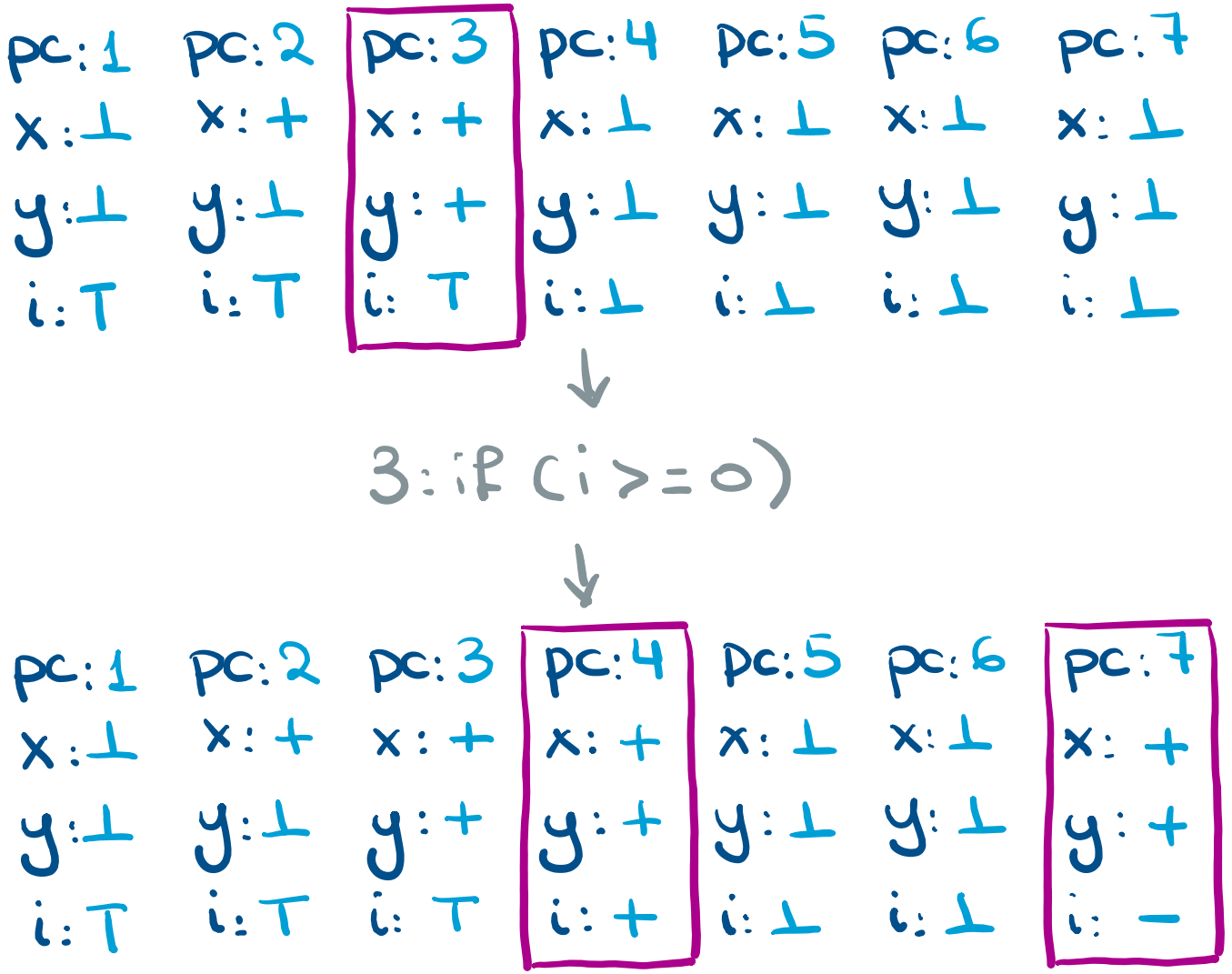
Note: We are essentially traversing a CFG with abstract states as nodes

Example – Step 3: Iterate to fixed point

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= x + y;
}
```

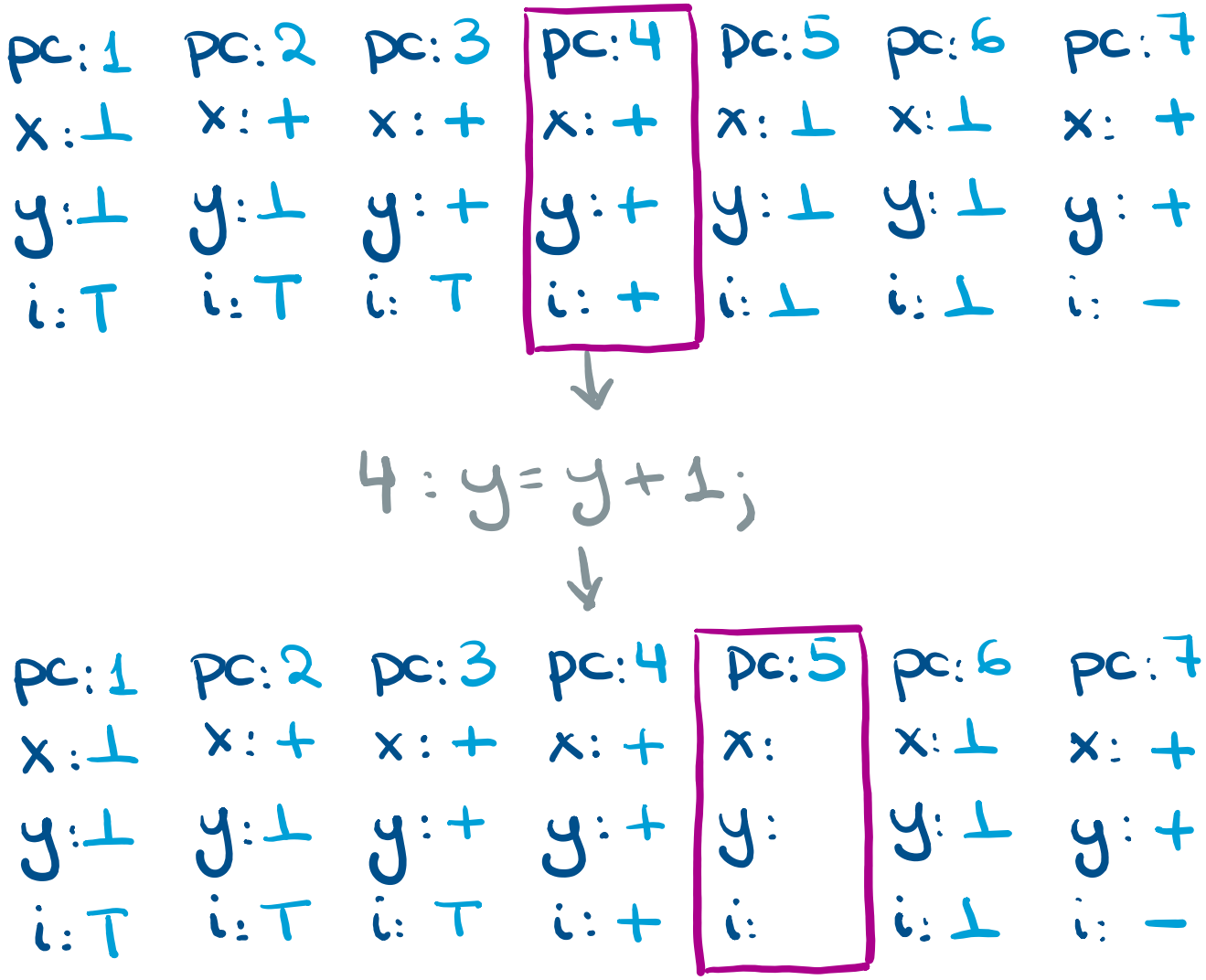


Example – Step 3: Iterate to fixed point

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= x + y;
}
```

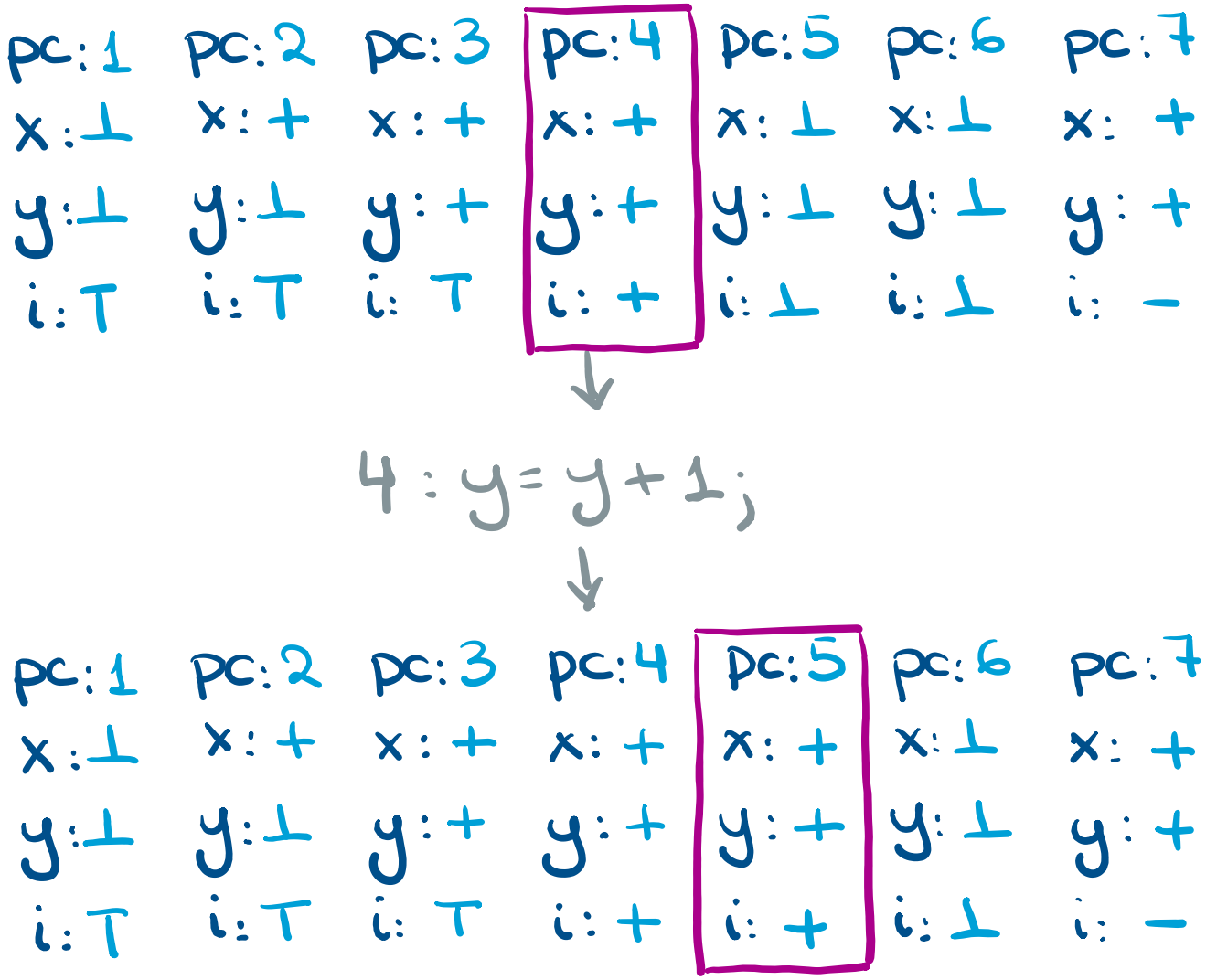


Example – Step 3: Iterate to fixed point

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= x + y;
}
```

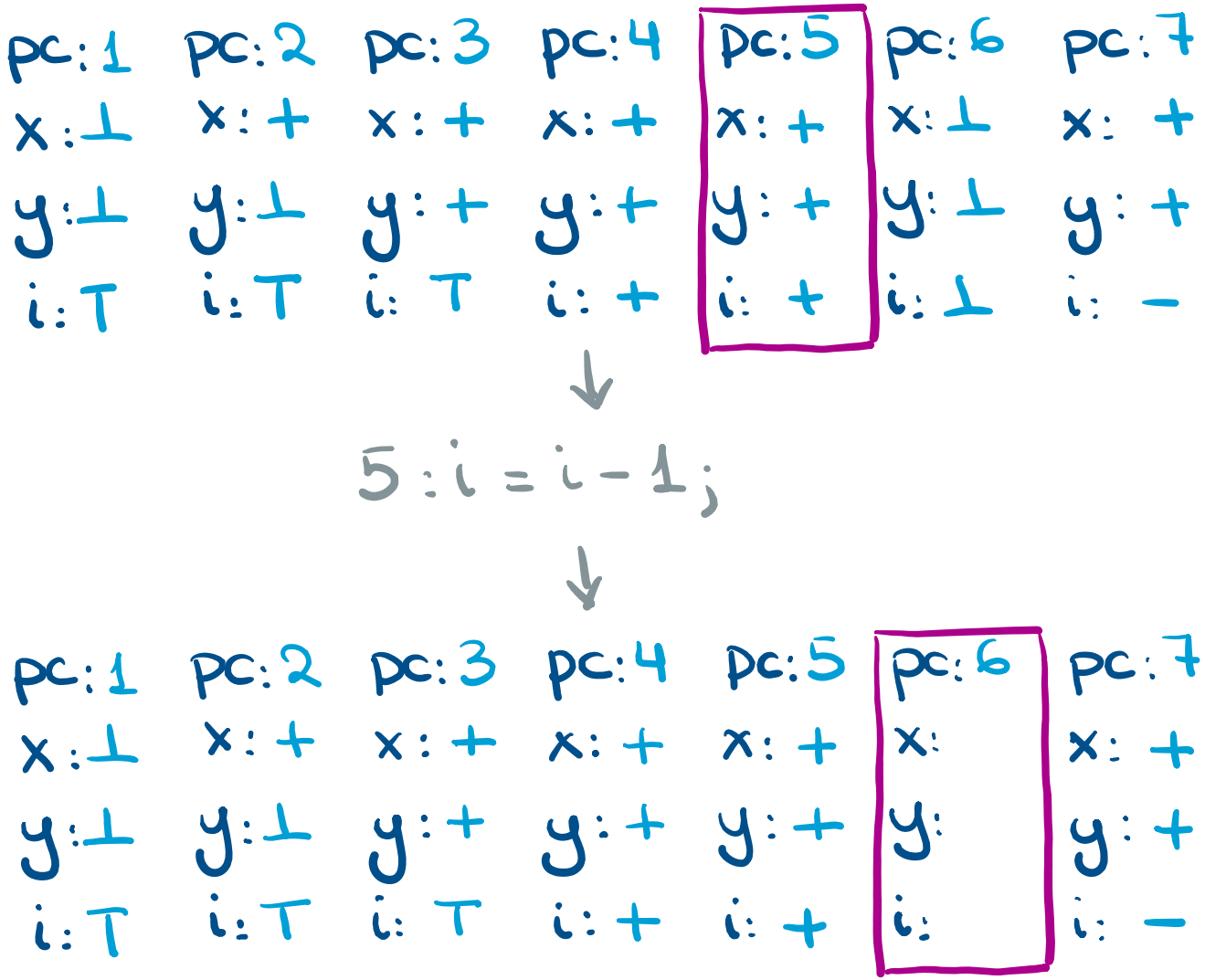


Example – Step 3: Iterate to fixed point

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= x + y;
}
```

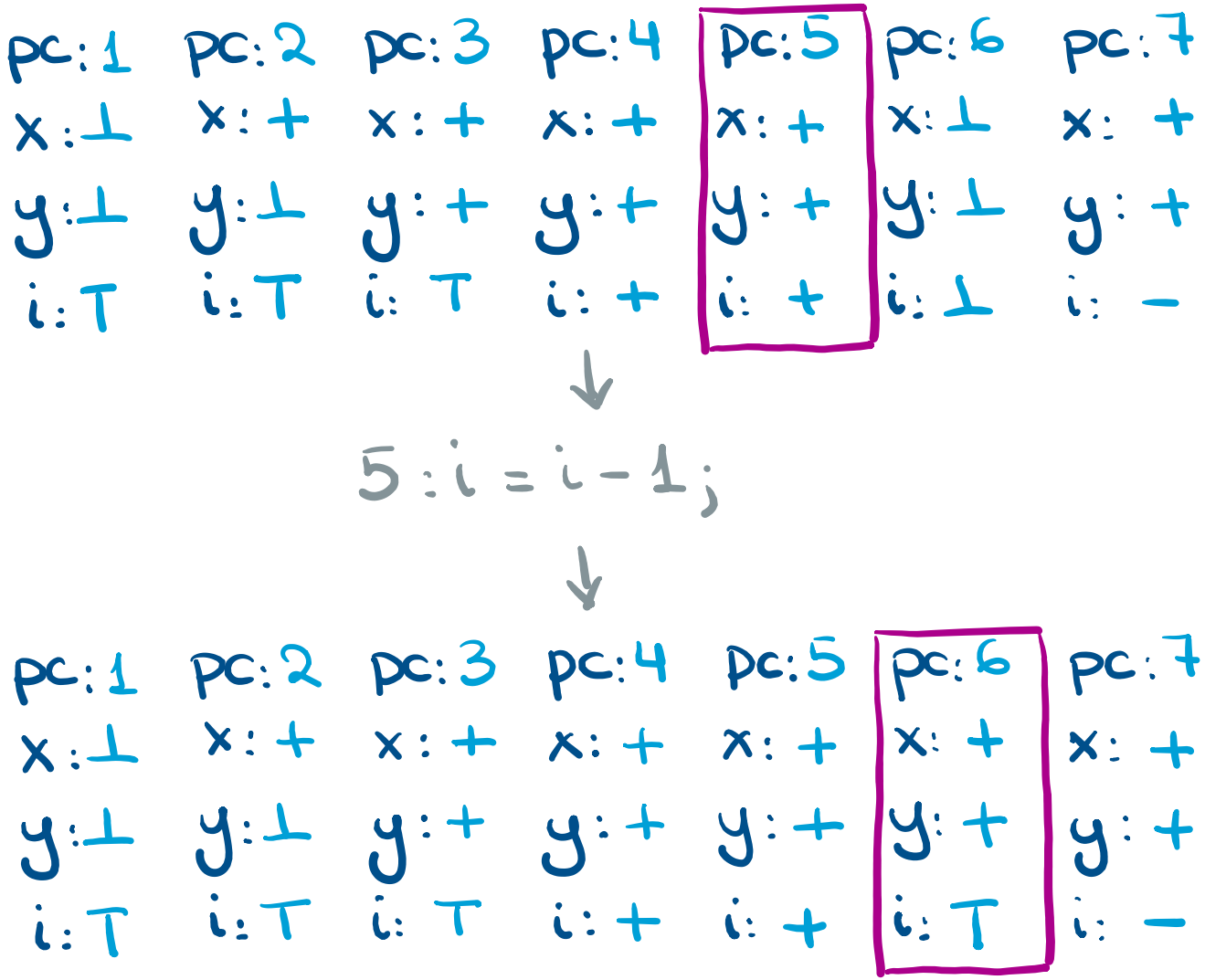


Example – Step 3: Iterate to fixed point

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= x + y;
}
```

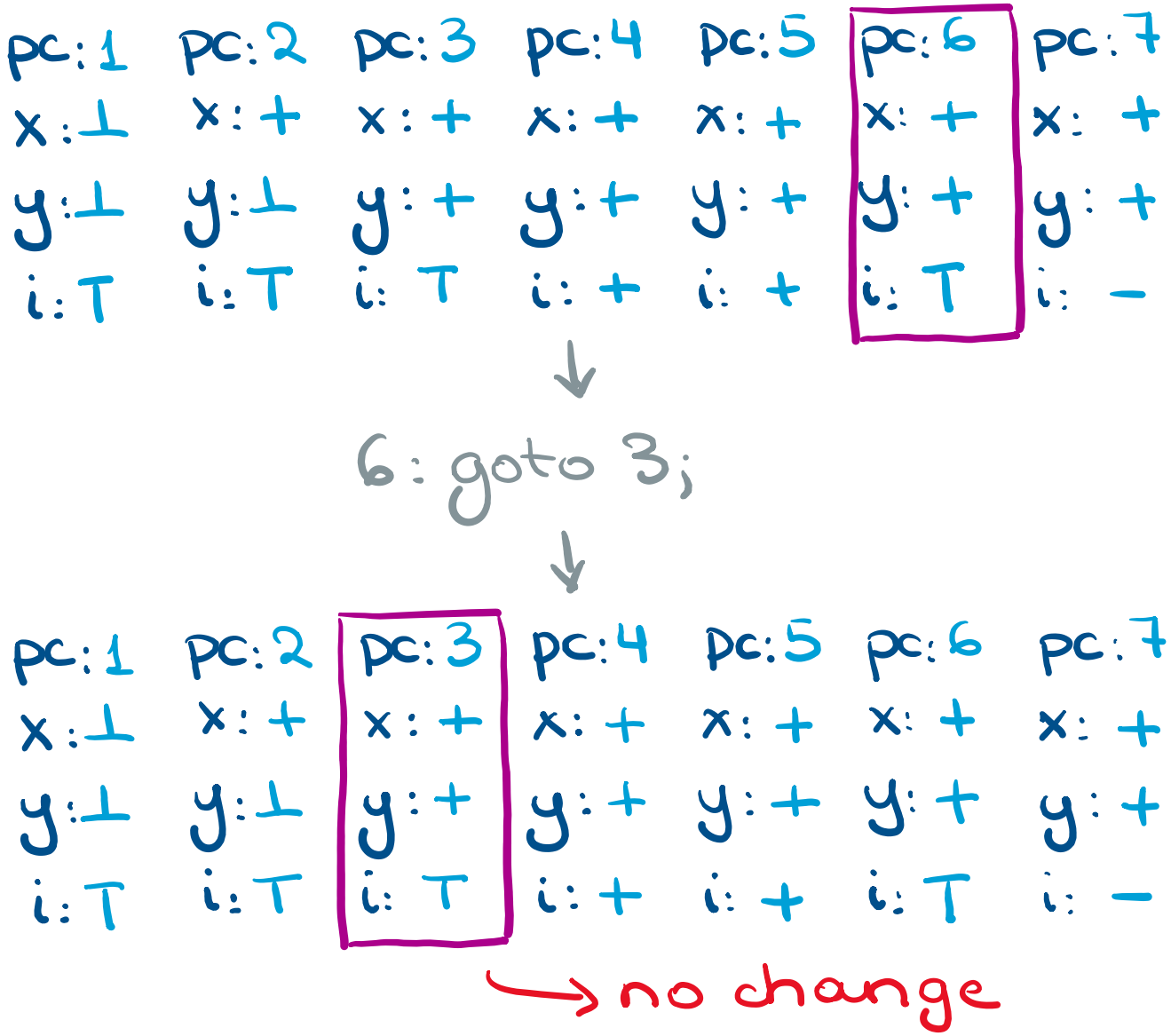


Example – Step 3: Iterate to fixed point

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= x + y;
}
```



Example – Step 3: Iterate to fixed point

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= x + y;
}
```

pc:1	pc:2	pc:3	pc:4	pc:5	pc:6	pc:7
x:⊥	x:+	x:+	x:+	x:+	x:+	x:+
y:⊥	y:⊥	y:+	y:+	y:+	y:+	y:+
i:⊤	i:⊤	i:⊤	i:+	i:+	i:⊤	i:−

We have reached a fixed point

Example – Step 3: Iterate to fixed point

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

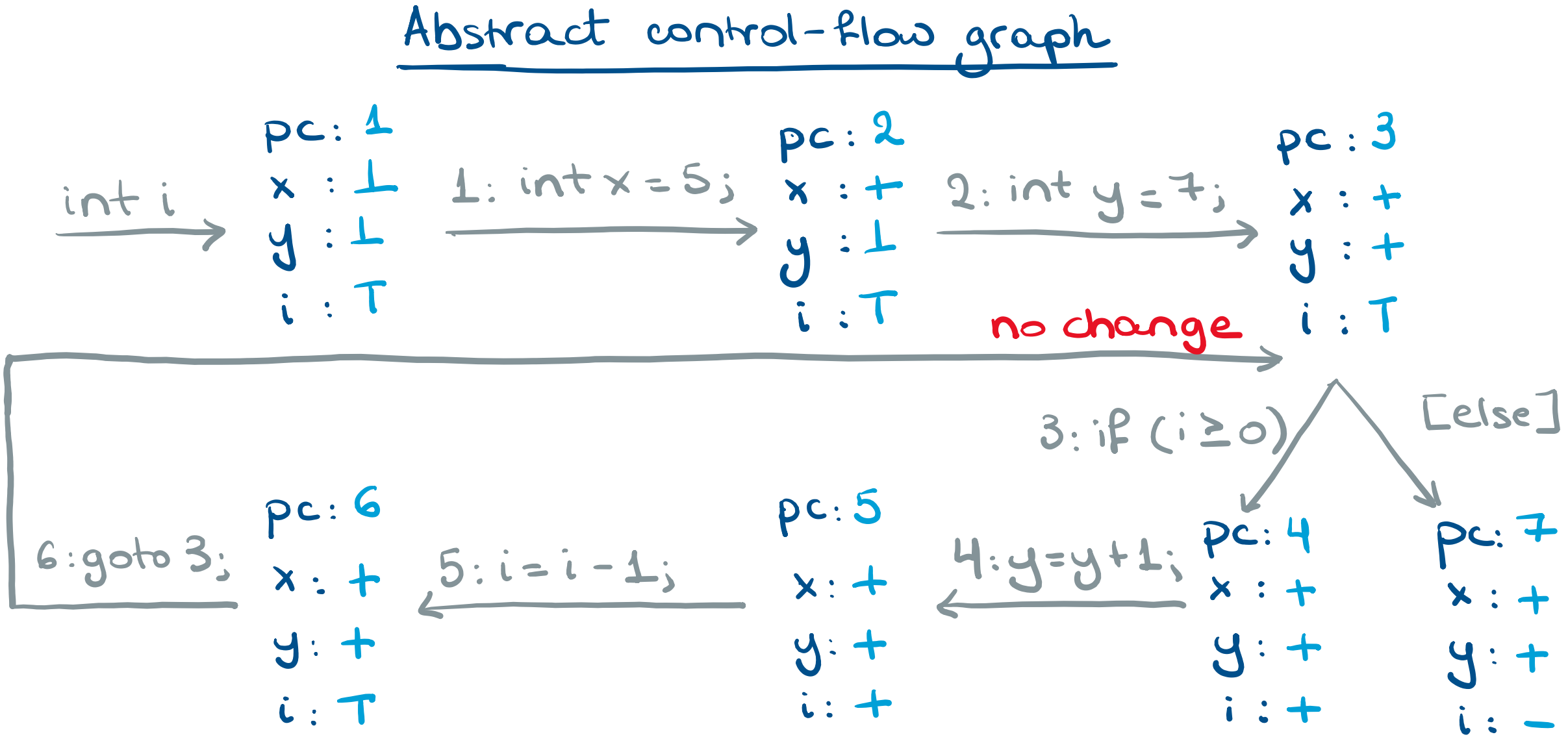
  7: assert 0 <= x + y;
}
```

pc:1	pc:2	pc:3	pc:4	pc:5	pc:6	pc:7
x:⊥	x:+	x:+	x:+	x:+	x:+	x:+
y:⊥	y:⊥	y:+	y:+	y:+	y:+	y:+
i:⊤	i:⊤	i:⊤	i:+	i:+	i:⊤	i:−

We have reached a fixed point

This is the program over-approximation and the program's invariants

Example – Step 3: Iterate to fixed point

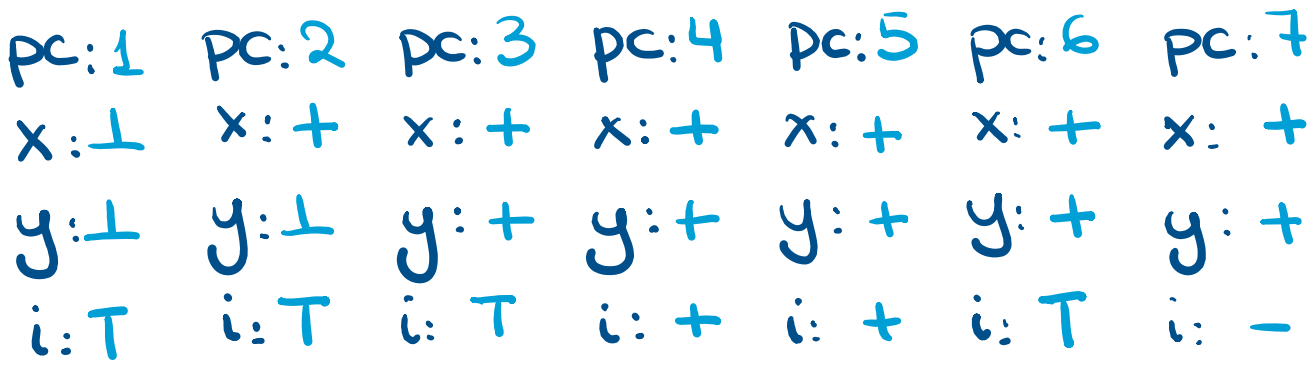


Invariants

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= x + y;
}
```



- Invariants are conditions that always hold at a particular program point
- In this example, $x \geq 0$ is always true at point 2
- So, abstract interpretation automatically infers invariants for a given program

Example – Step 4: Check property

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= x + y;
}
```

pc:1	pc:2	pc:3	pc:4	pc:5	pc:6	pc:7
x:⊥	x:+	x:+	x:+	x:+	x:+	x:+
y:⊥	y:⊥	y:+	y:+	y:+	y:+	y:+
i:⊤	i:⊤	i:⊤	i:+	i:+	i:⊤	i:−

$P \models (0 \leq x + y)$ ✓

$P_{sign} \models (0 \leq x + y)$ ✓

The sign domain is precise enough to prove the property

Example – Step 4: Check property

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= x - y;
}
```

pc:1	pc:2	pc:3	pc:4	pc:5	pc:6	pc:7
x:⊥	x:+	x:+	x:+	x:+	x:+	x:+
y:⊥	y:⊥	y:+	y:+	y:+	y:+	y:+
i:T	i:T	i:T	i:+	i:+	i:T	i:−

$P \quad (0 \leq x - y)$

$P_{sign} \quad (0 \leq x - y)$

Example – Step 4: Check property

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= x - y;
}
```

pc:1	pc:2	pc:3	pc:4	pc:5	pc:6	pc:7
x:⊥	x:+	x:+	x:+	x:+	x:+	x:+
y:⊥	y:⊥	y:+	y:+	y:+	y:+	y:+
i:T	i:T	i:T	i:+	i:+	i:T	i:−

$P \not\models (0 \leq x - y)$ ✗

$P_{sign} \quad (0 \leq x - y)$

Example – Step 4: Check property

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= x - y;
}
```

pc:1	pc:2	pc:3	pc:4	pc:5	pc:6	pc:7
x:⊥	x:+	x:+	x:+	x:+	x:+	x:+
y:⊥	y:⊥	y:+	y:+	y:+	y:+	y:+
i:T	i:T	i:T	i:+	i:+	i:T	i:−

$P \not\models (0 \leq x - y)$ ✗

$P_{sign} \not\models (0 \leq x - y)$ ✗

Example – Step 4: Check property

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= x - y;
}
```

pc:1	pc:2	pc:3	pc:4	pc:5	pc:6	pc:7
x:⊥	x:+	x:+	x:+	x:+	x:+	x:+
y:⊥	y:⊥	y:+	y:+	y:+	y:+	y:+
i:T	i:T	i:T	i:+	i:+	i:T	i:−

$P \not\models (0 \leq x - y)$ ✗

$P_{sign} \not\models (0 \leq x - y)$ ✗

The sign domain is sound:
the property does not hold and it confirms it

Example – Step 4: Check property

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= y - x;
}
```

pc:1	pc:2	pc:3	pc:4	pc:5	pc:6	pc:7
x:⊥	x:+	x:+	x:+	x:+	x:+	x:+
y:⊥	y:⊥	y:+	y:+	y:+	y:+	y:+
i:T	i:T	i:T	i:+	i:+	i:T	i:−

$P \quad (0 \leq y - x)$

$P_{sign} \quad (0 \leq y - x)$

Example – Step 4: Check property

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= y - x;
}
```

pc: 1	pc: 2	pc: 3	pc: 4	pc: 5	pc: 6	pc: 7
x: ⊥	x: +	x: +	x: +	x: +	x: +	x: +
y: ⊥	y: ⊥	y: +	y: +	y: +	y: +	y: +
i: T	i: T	i: T	i: +	i: +	i: T	i: −

$P \models (0 \leq y - x)$ ✓

$P_{sign} \quad (0 \leq y - x)$

Example – Step 4: Check property

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= y - x;
}
```

pc:1	pc:2	pc:3	pc:4	pc:5	pc:6	pc:7
x:⊥	x:+	x:+	x:+	x:+	x:+	x:+
y:⊥	y:⊥	y:+	y:+	y:+	y:+	y:+
i:T	i:T	i:T	i:+	i:+	i:T	i:−

$P \models (0 \leq y - x)$ ✓

$P_{sign} \not\models (0 \leq y - x)$ ✗

Example – Step 4: Check property

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= y - x;
}
```

pc:1	pc:2	pc:3	pc:4	pc:5	pc:6	pc:7
x:⊥	x:+	x:+	x:+	x:+	x:+	x:+
y:⊥	y:⊥	y:+	y:+	y:+	y:+	y:+
i:⊤	i:⊤	i:⊤	i:+	i:+	i:⊤	i:−

$P \models (0 \leq y - x)$ ✓

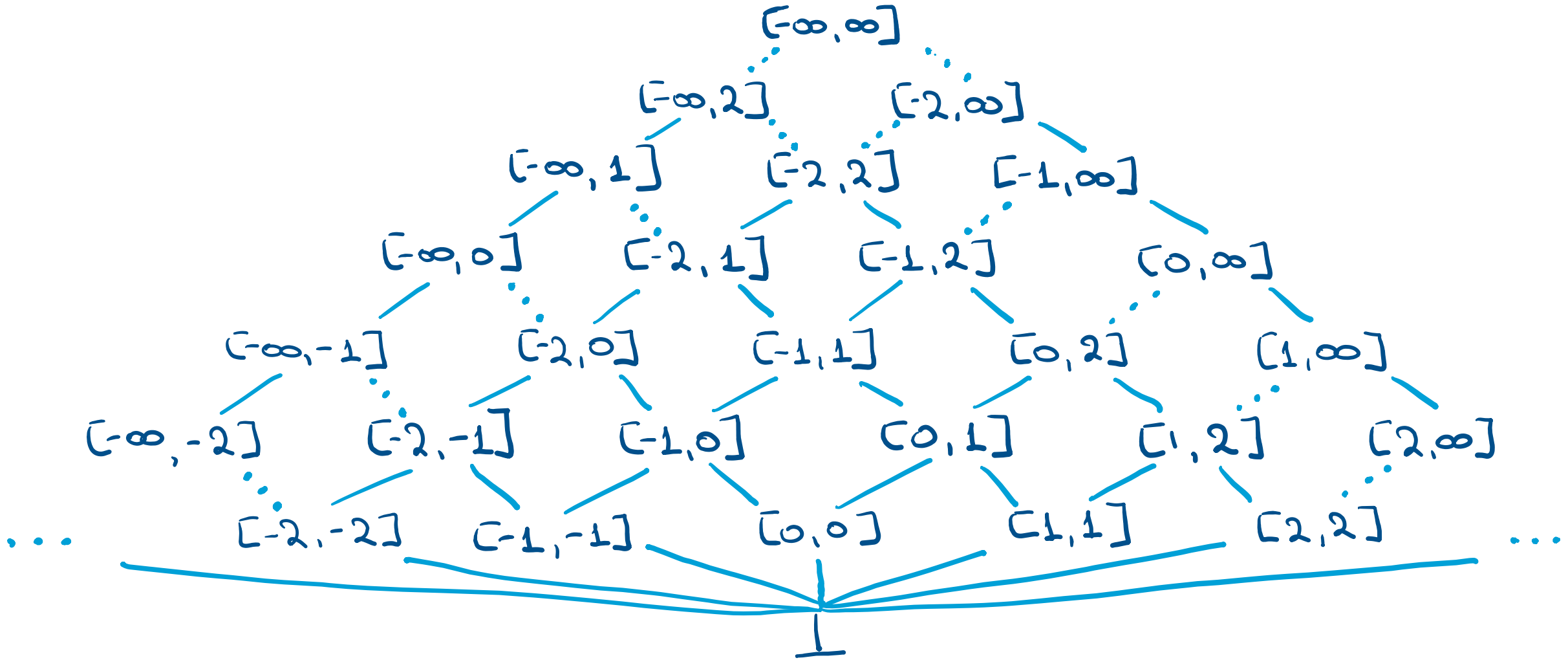
$P_{sign} \not\models (0 \leq y - x)$ ✗

The sign domain is too imprecise to prove the property

The interval domain

- The interval abstract domain, instead of abstracting variable values using the sign of variables, abstracts variable values using an interval

The interval domain



Example – Step 1: Select abstraction

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i – 1;
  6:   goto 3;
     }

  7: assert 0 <= y - x;
}
```

- Let's pick the interval abstract domain

An abstract program state now looks like

pc: 4

x: [-2, ∞]

y: [1, 7]

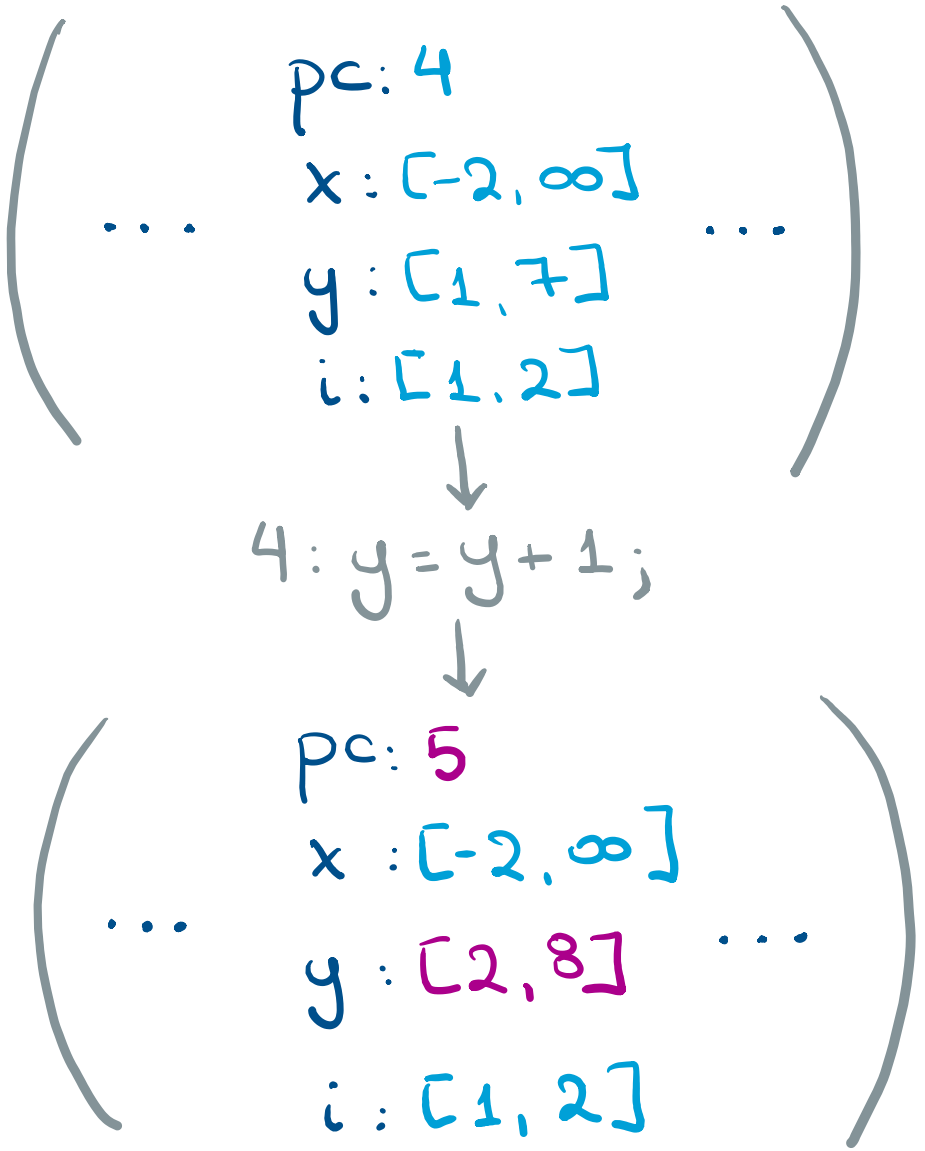
i: [1, 2]

Example – Step 2: Define transformers

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i – 1;
  6:   goto 3;
     }

  7: assert 0 <= y - x;
}
```

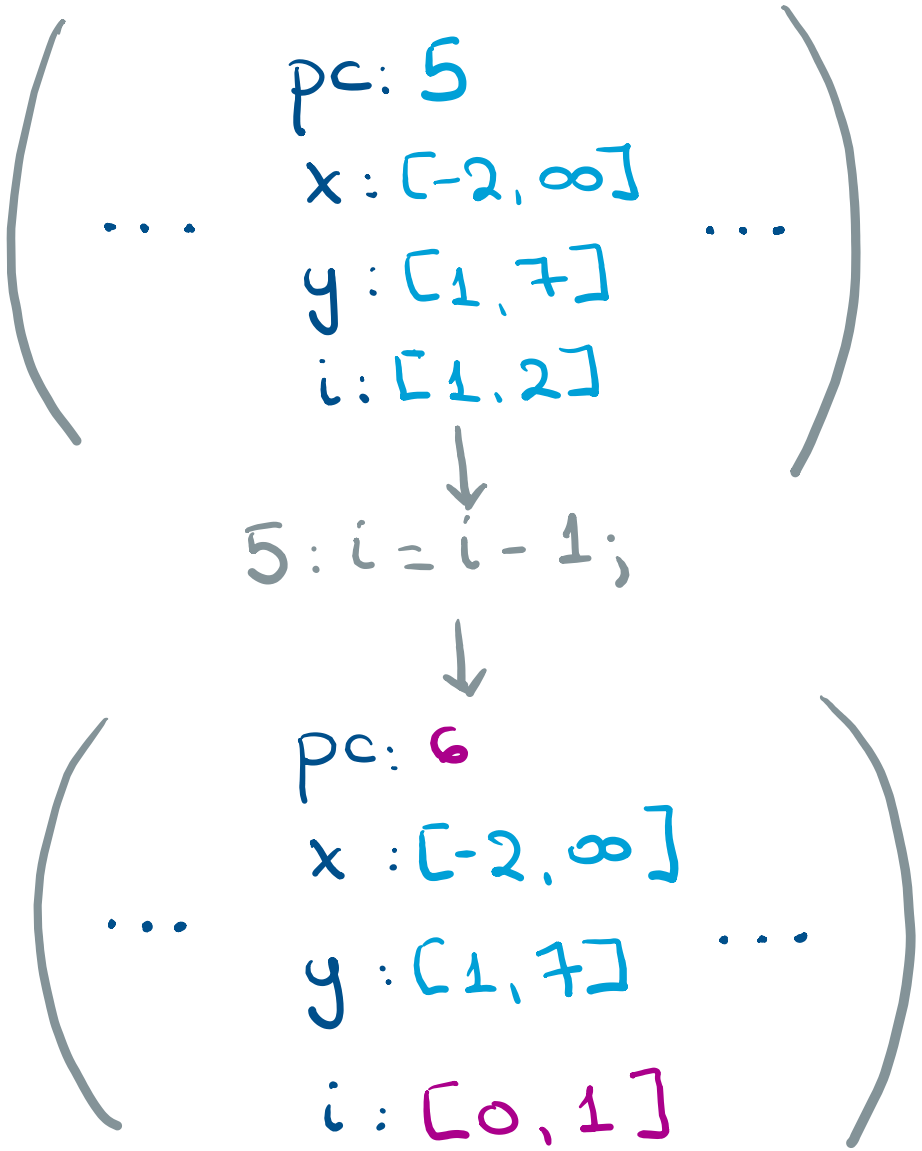


Example – Step 2: Define transformers

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i – 1;
  6:   goto 3;
     }

  7: assert 0 <= y - x;
}
```



Example – Step 3: Iterate to fixed point

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i – 1;
  6:   goto 3;
     }

  7: assert 0 <= y - x;
}
```

Again, start with the least abstract element

pc: 1
x: ⊥
y: ⊥
i: ⊥

pc: 2
x: ⊥
y: ⊥
i: ⊥

pc: 3
x: ⊥
y: ⊥
i: ⊥

pc: 4
x: ⊥
y: ⊥
i: ⊥

pc: 5
x: ⊥
y: ⊥
i: ⊥

pc: 6
x: ⊥
y: ⊥
i: ⊥

pc: 7
x: ⊥
y: ⊥
i: ⊥

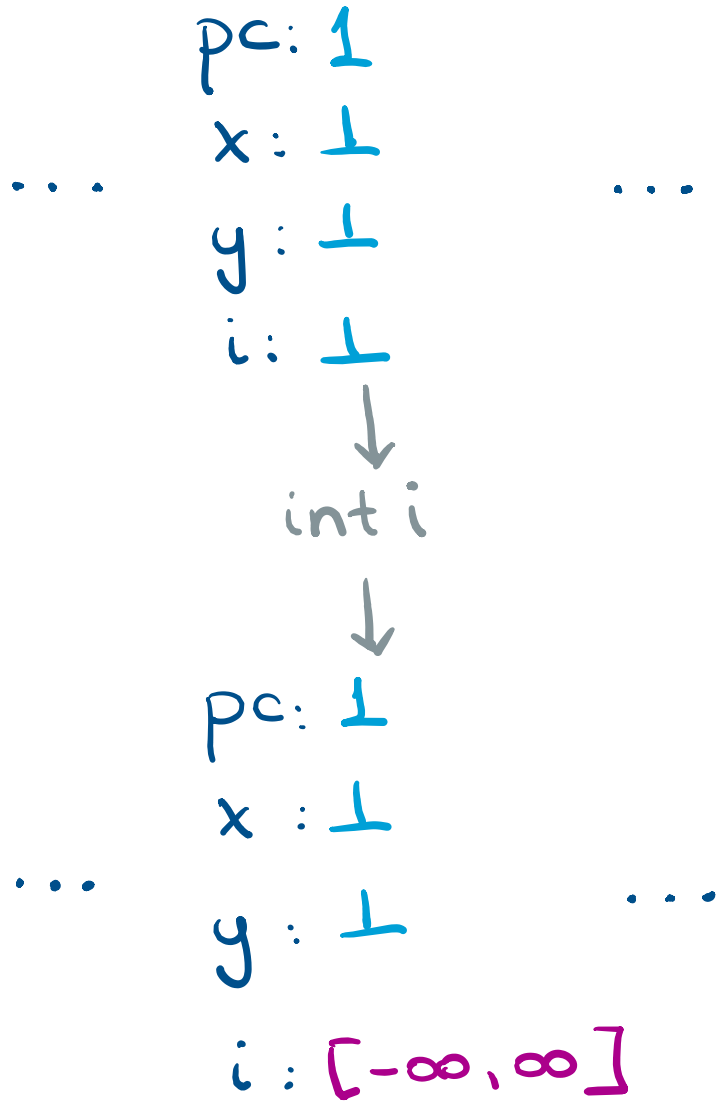
Let's do some iterations!

Example – Step 3: Iterate to fixed point

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i – 1;
  6:   goto 3;
     }

  7: assert 0 <= y - x;
}
```

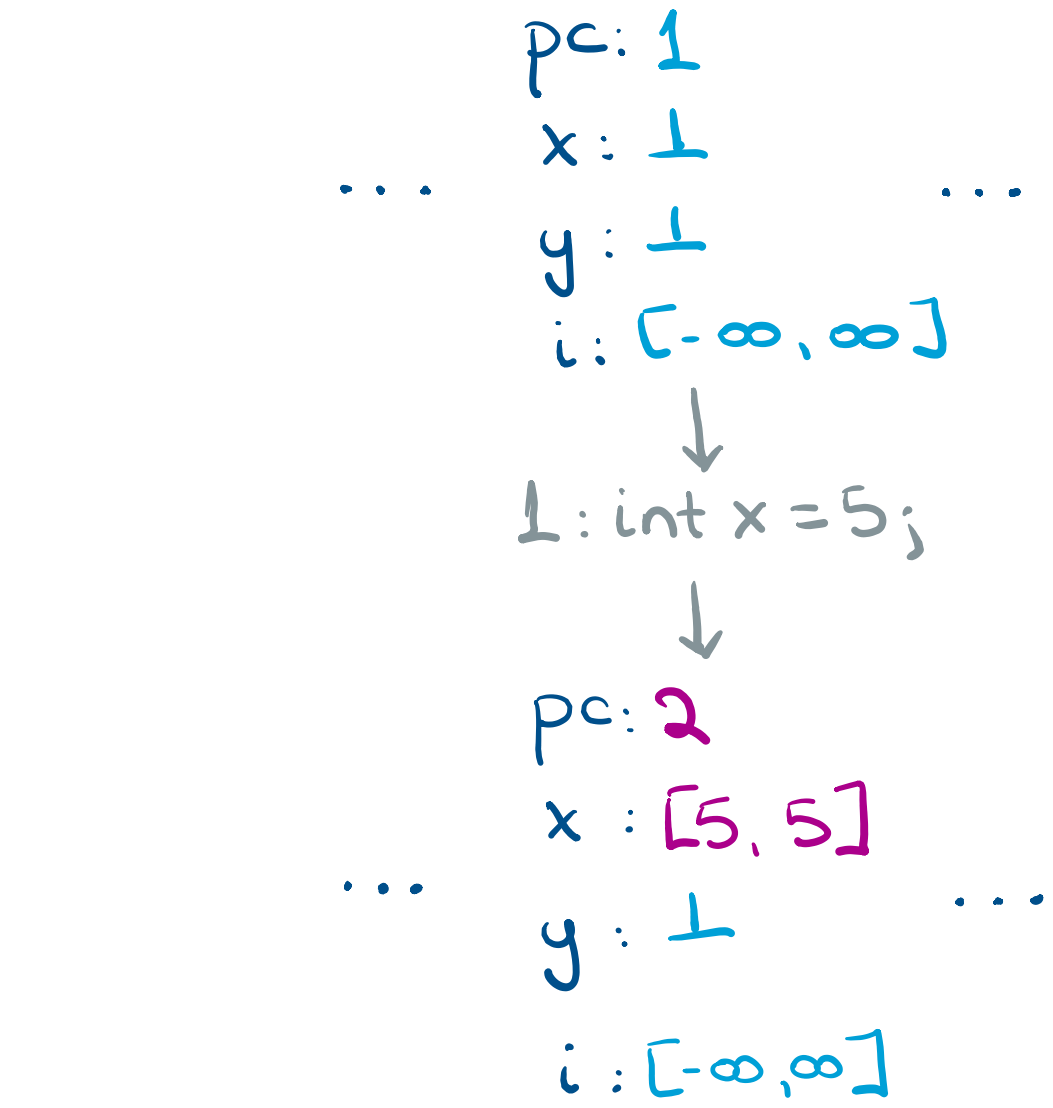


Example – Step 3: Iterate to fixed point

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i – 1;
  6:   goto 3;
     }

  7: assert 0 <= y - x;
}
```

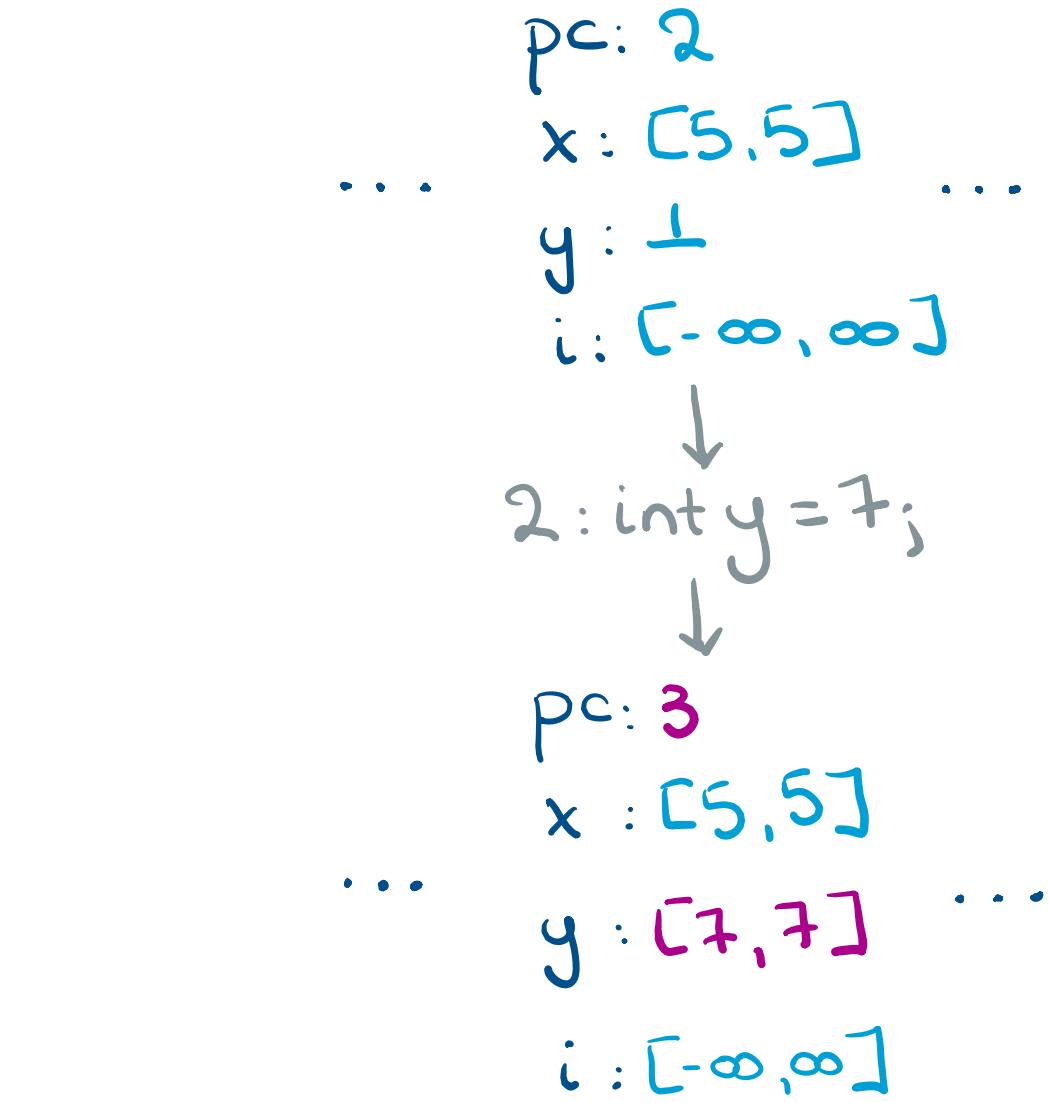


Example – Step 3: Iterate to fixed point

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i – 1;
  6:   goto 3;
     }

  7: assert 0 <= y - x;
}
```

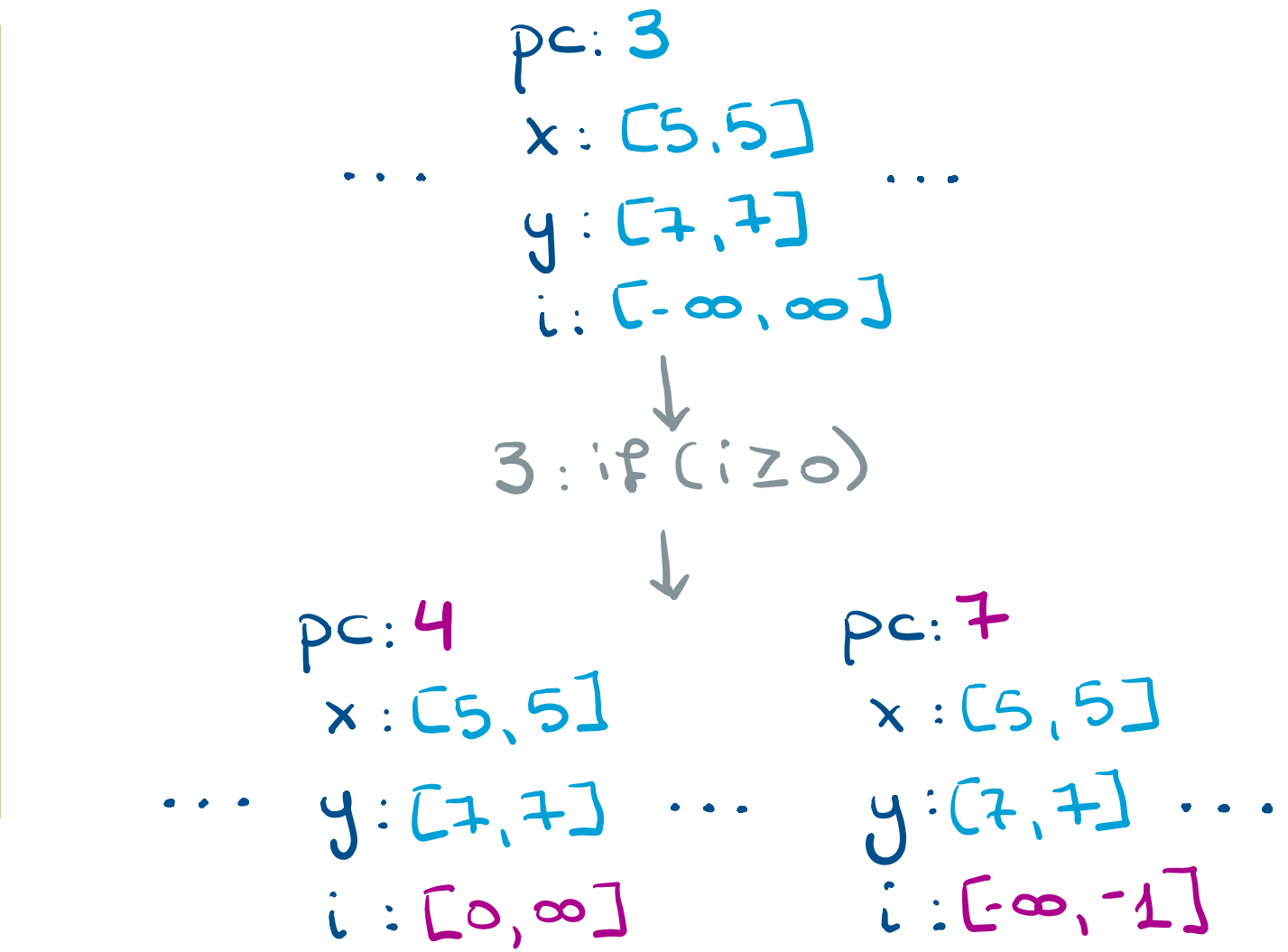


Example – Step 3: Iterate to fixed point

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i – 1;
  6:   goto 3;
     }

  7: assert 0 <= y - x;
}
```

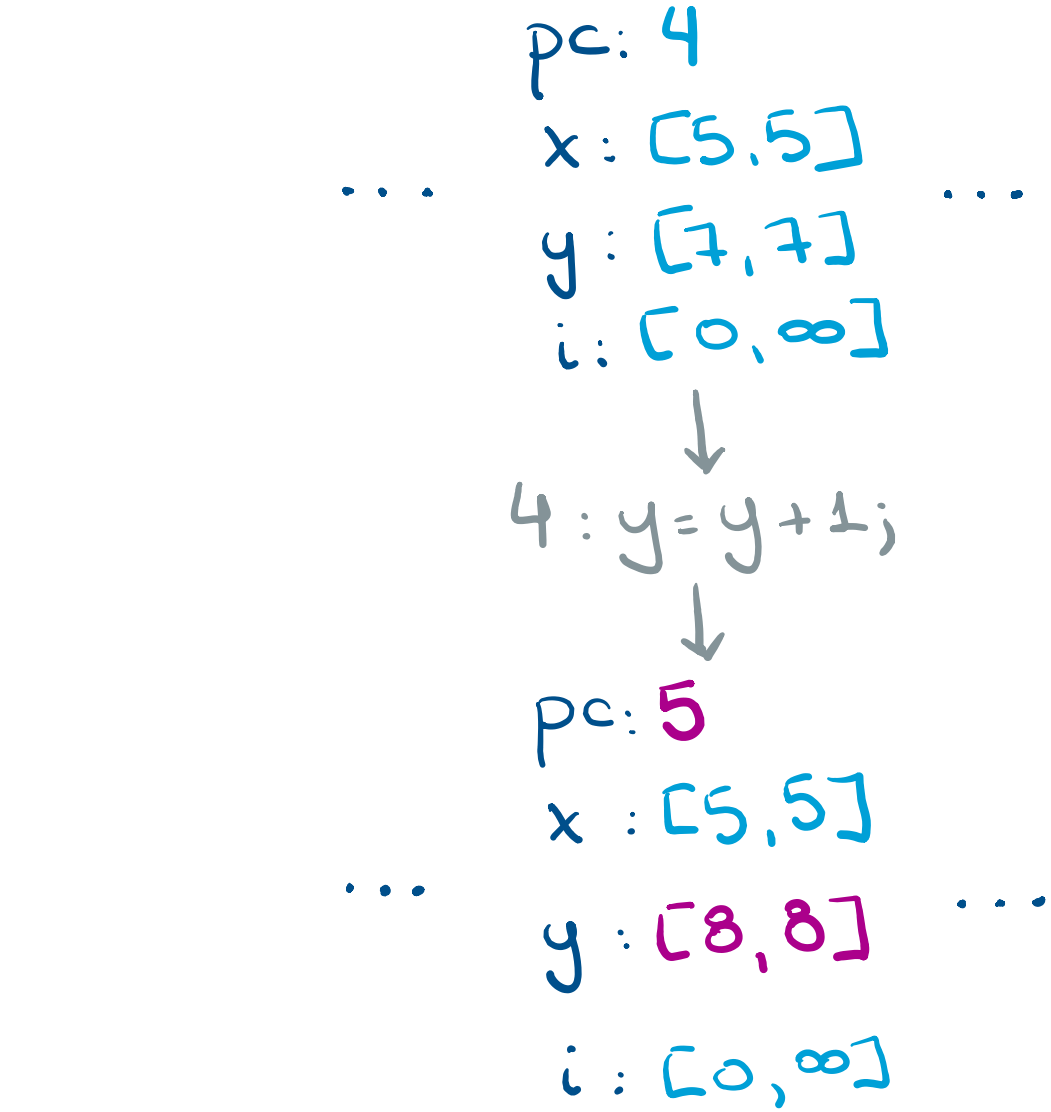


Example – Step 3: Iterate to fixed point

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i – 1;
  6:   goto 3;
     }

  7: assert 0 <= y - x;
}
```

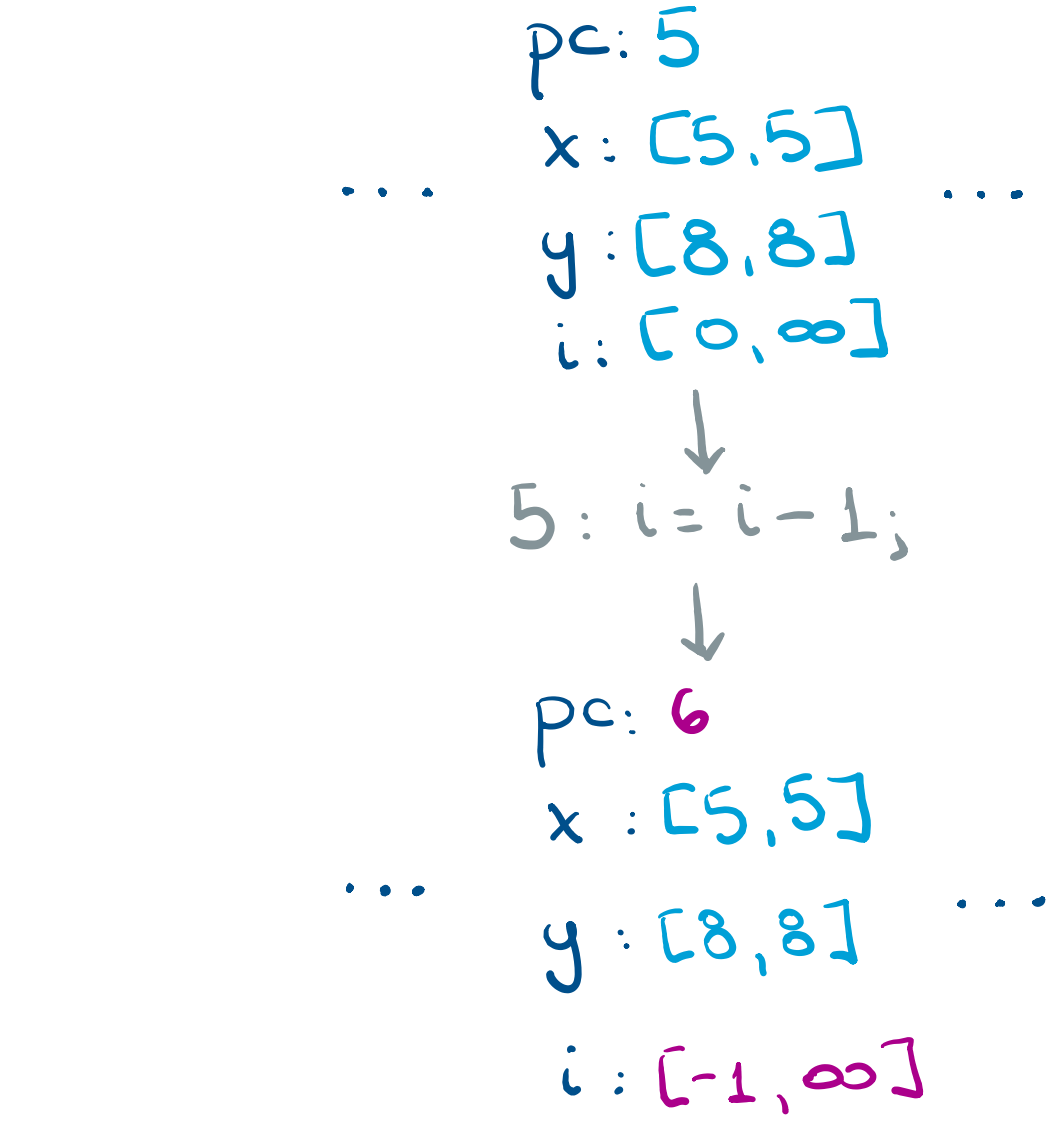


Example – Step 3: Iterate to fixed point

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i – 1;
  6:   goto 3;
     }

  7: assert 0 <= y - x;
}
```

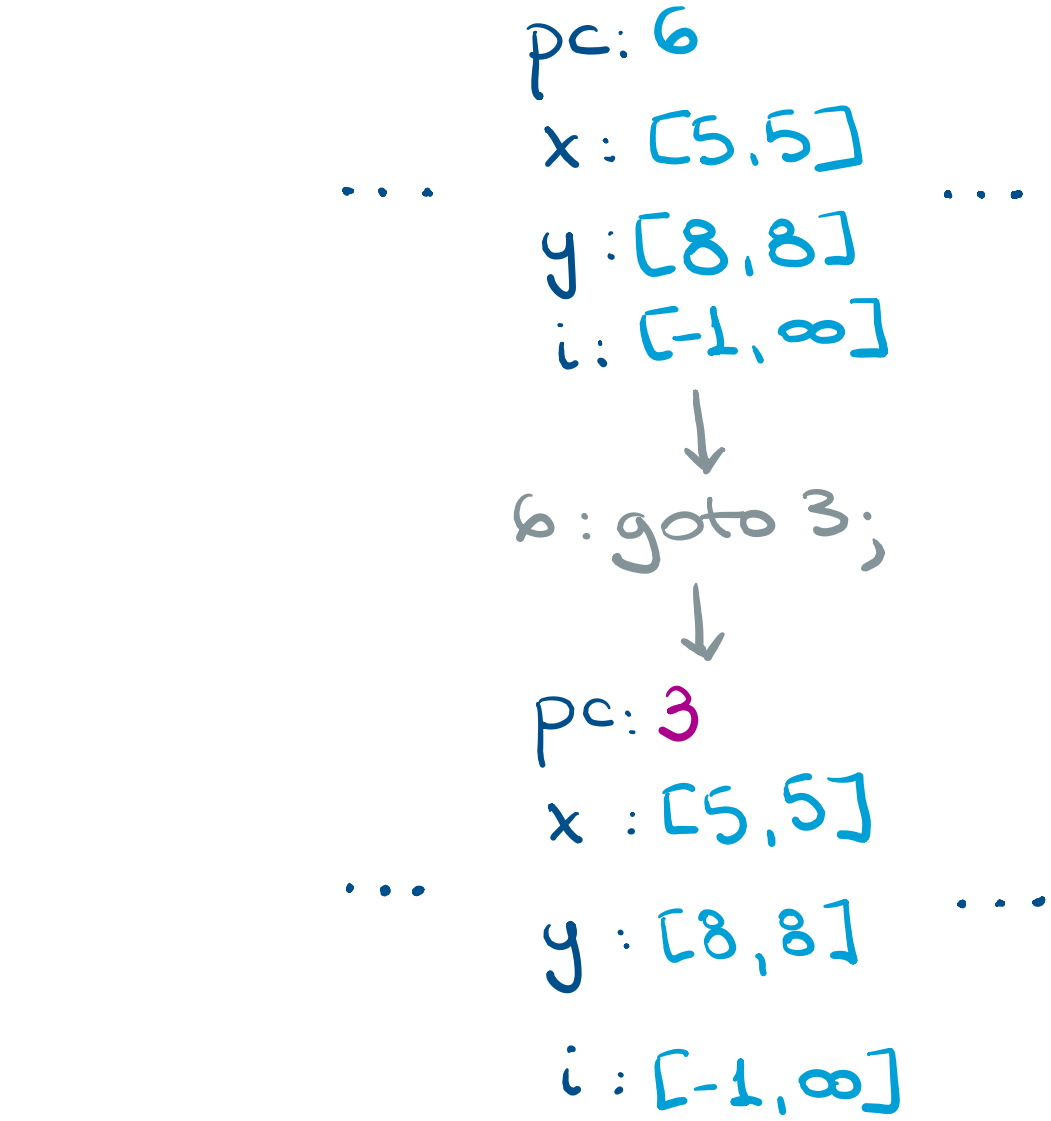


Example – Step 3: Iterate to fixed point

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i – 1;
  6:   goto 3;
     }

  7: assert 0 <= y - x;
}
```

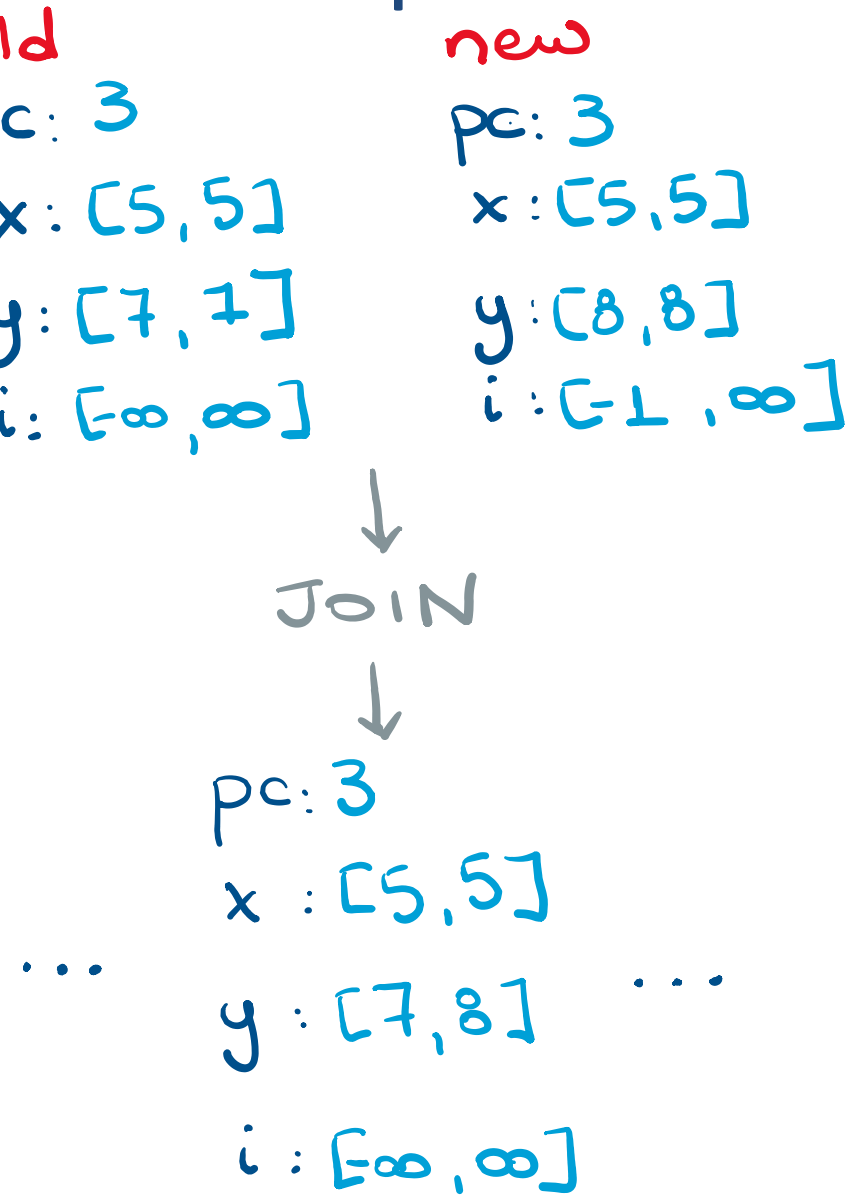


Example – Step 3: Iterate to fixed point

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i – 1;
  6:   goto 3;
     }

  7: assert 0 <= y - x;
}
```



Join

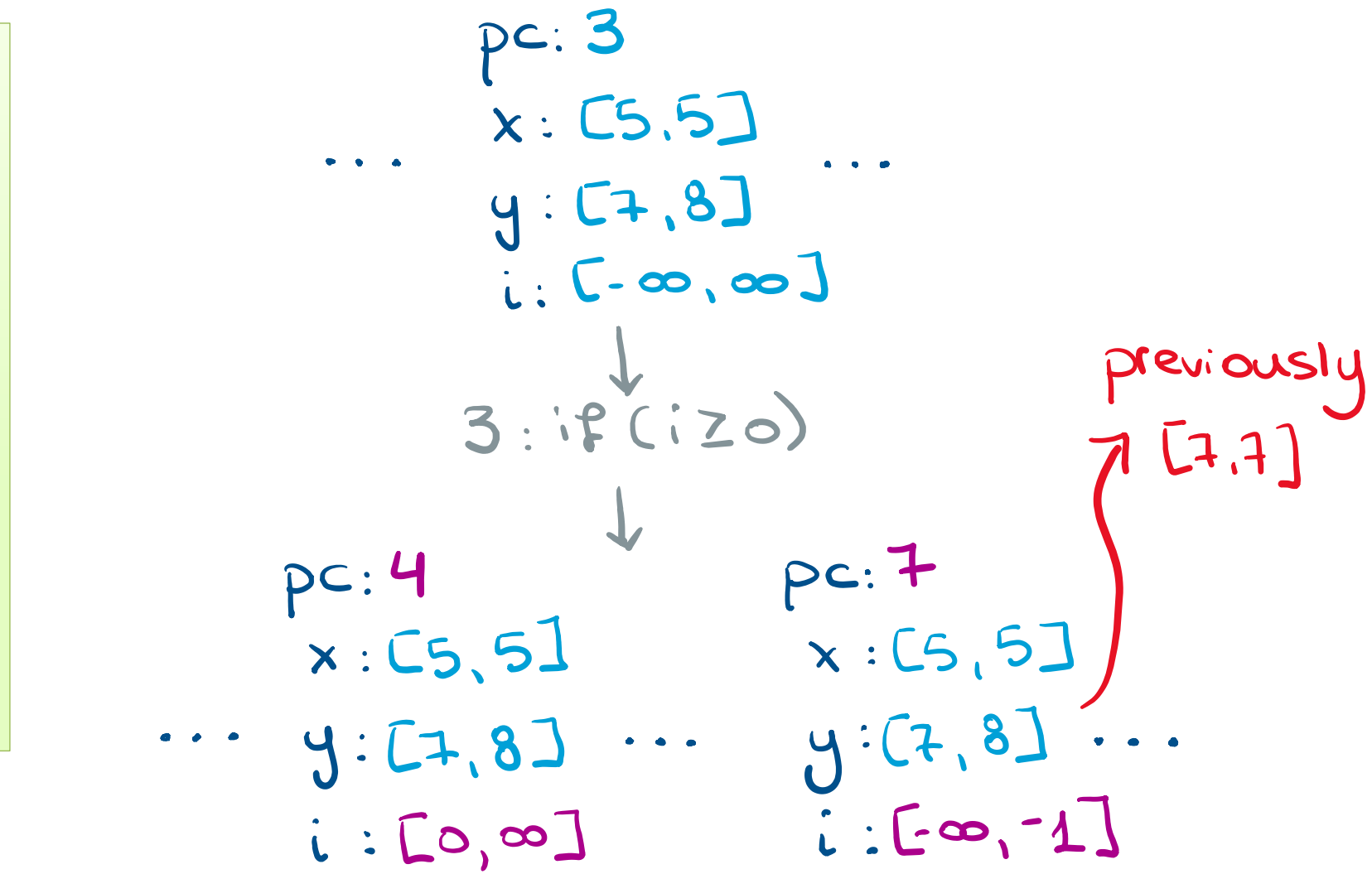
- When we have two abstract elements A and B, we can join them to produce their (least) upper bound
- The join is denoted by $A \sqcup B$
- We have that $A \sqsubseteq A \sqcup B$ and $B \sqsubseteq A \sqcup B$
  - $C \sqsubseteq D$ means that D is more abstract than C
- In our example, we join the abstract states that occur at the same program label

Example – Step 3: Iterate to fixed point

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= y - x;
}
```

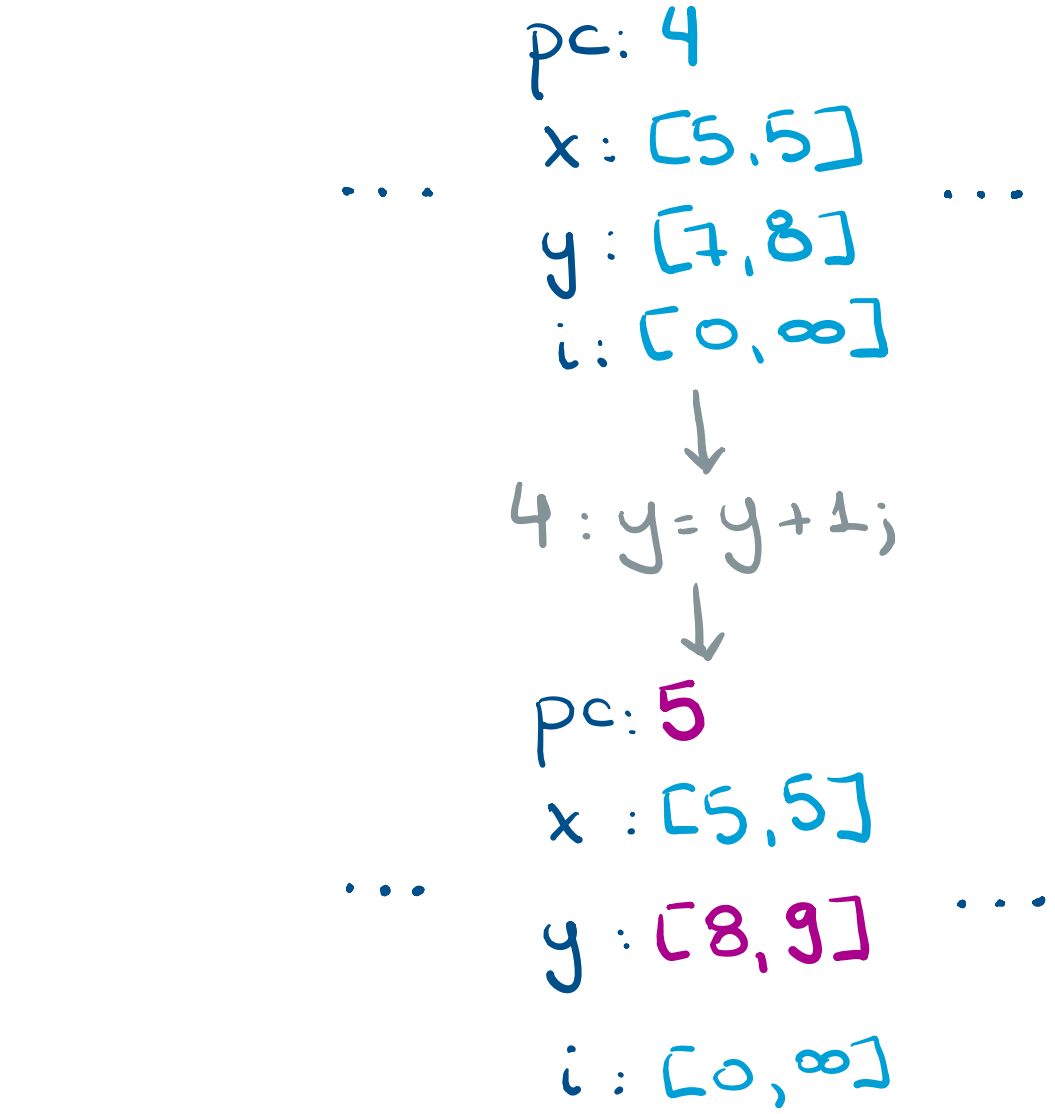


Example – Step 3: Iterate to fixed point

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i – 1;
  6:   goto 3;
     }

  7: assert 0 <= y - x;
}
```

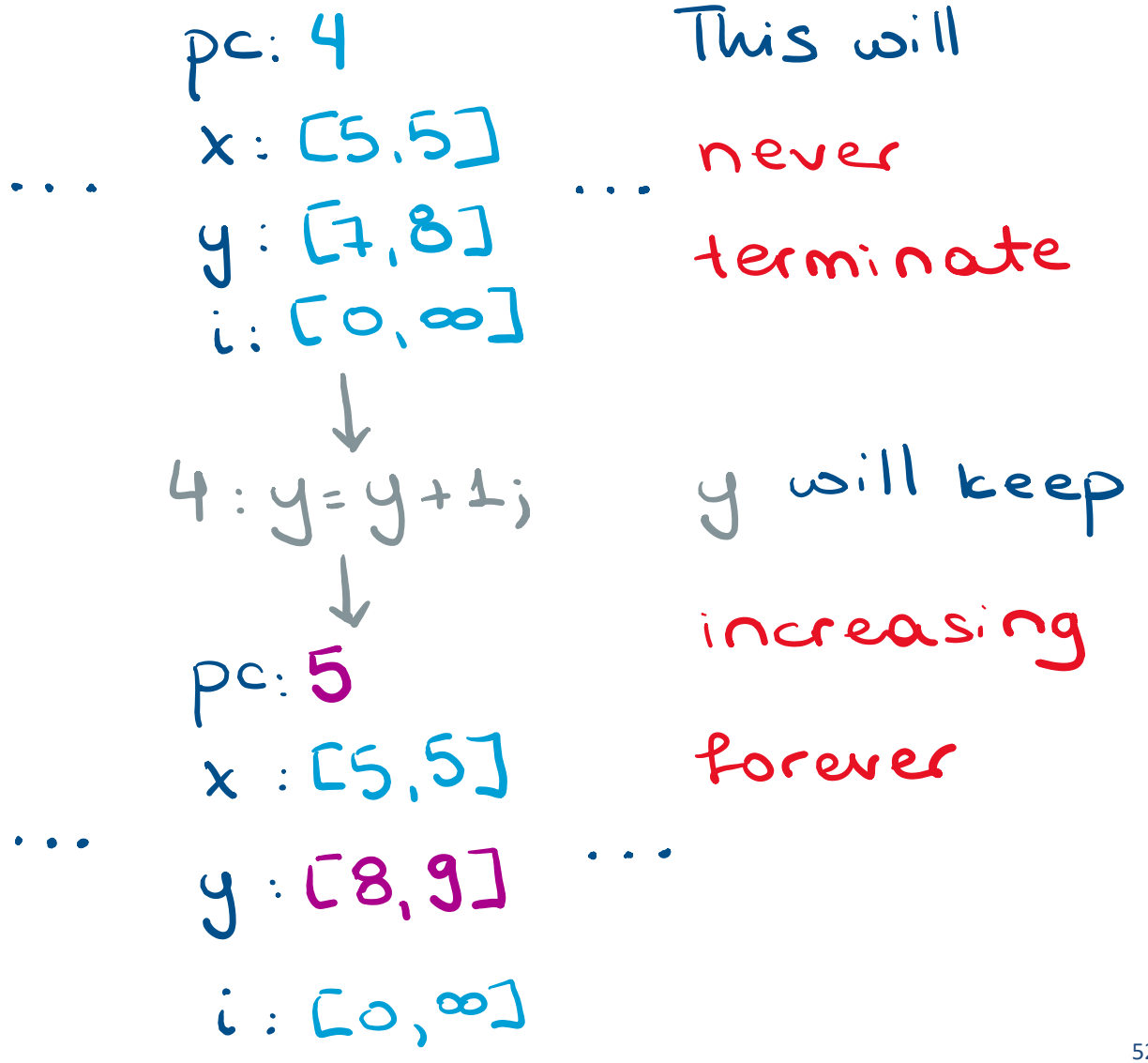


Example – Step 3: Iterate to fixed point

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i – 1;
  6:   goto 3;
     }

  7: assert 0 <= y - x;
}
```



Example – Step 3: Iterate to fixed point

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i – 1;
  6:   goto 3;
     }

  7: assert 0 <= y - x;
}
```

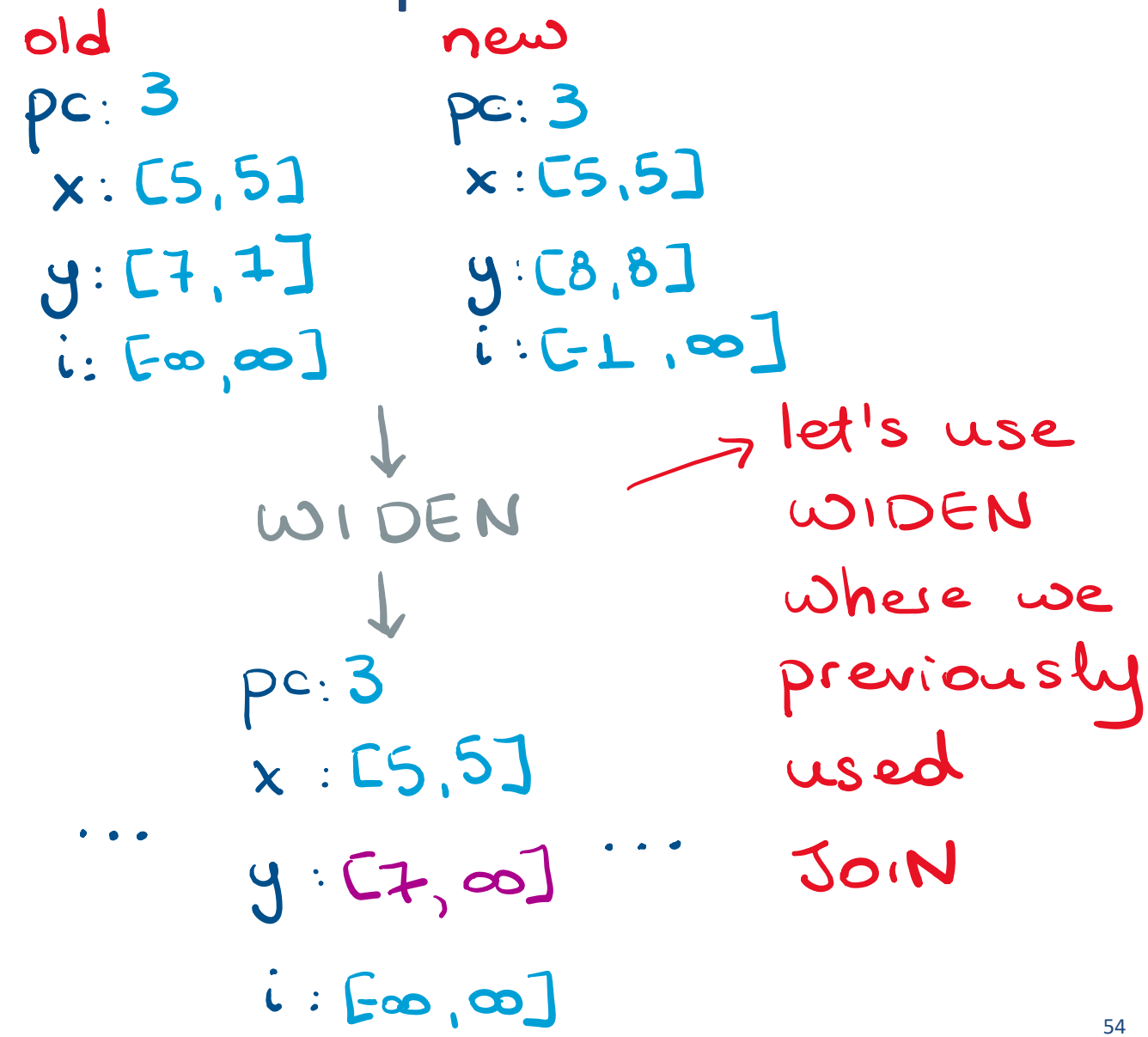
- The interval domain has infinite height
- We cannot reach a fixed point with the interval domain
- For this reason, we introduce a special operator, called widening
- Widening ensures termination at the expense of precision

Example – Step 3: Iterate to fixed point

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= y - x;
}
```



Widening

- When the abstract domain does not have finite height, we need the widening operator for convergence
  - When to apply widening depends on the analysis, typically for loops
- The sign domain has the finite-height property
- The interval domain does not have the finite-height property
- For the interval domain, we can define the following widening operator

$$[a,b] \nabla \bot = [a,b] \quad \text{and} \quad \bot \nabla [a,b] = [a,b]$$

$$[a,b] \nabla [c,d] = [c < a \,?\, -\infty : a,\ b < d \,?\, +\infty : b]$$

Widening: Exercise

$[1,2] \nabla [0,2] =$

$[0,2] \nabla [1,2] =$

$[1,5] \nabla [1,5] =$

$[2,3] \nabla [2,4] =$

Definition:

$[a,b] \nabla \bot = [a,b]$ and $\bot \nabla [a,b] = [a,b]$

$[a,b] \nabla [c,d] = [c < a ? -\infty : a, b < d ? +\infty : b]$

Widening: Exercise

$[1,2] \nabla [0,2] = [-\infty, 2]$

$[0,2] \nabla [1,2] =$

$[1,5] \nabla [1,5] =$

$[2,3] \nabla [2,4] =$

Definition:

$[a,b] \nabla \bot = [a,b]$ and $\bot \nabla [a,b] = [a,b]$

$[a,b] \nabla [c,d] = [c < a ? -\infty : a, b < d ? +\infty : b]$

Widening: Exercise

$[1,2] \nabla [0,2] = [-\infty, 2]$

$[0,2] \nabla [1,2] = [0,2]$

$[1,5] \nabla [1,5] =$

$[2,3] \nabla [2,4] =$

Definition:

$[a,b] \nabla \bot = [a,b]$ and $\bot \nabla [a,b] = [a,b]$

$[a,b] \nabla [c,d] = [c < a ? -\infty : a, b < d ? +\infty : b]$

Widening: Exercise

$[1,2] \nabla [0,2] = [-\infty, 2]$

$[0,2] \nabla [1,2] = [0,2]$

Widening is NOT commutative

$[1,5] \nabla [1,5] =$

$[2,3] \nabla [2,4] =$

Definition:

$[a,b] \nabla \bot = [a,b]$ and $\bot \nabla [a,b] = [a,b]$

$[a,b] \nabla [c,d] = [c < a ? -\infty : a, b < d ? +\infty : b]$

Widening: Exercise

$[1,2] \nabla [0,2] = [-\infty, 2]$

$[0,2] \nabla [1,2] = [0,2]$

$[1,5] \nabla [1,5] = [1,5]$

$[2,3] \nabla [2,4] =$

Definition:

$[a,b] \nabla \bot = [a,b]$ and $\bot \nabla [a,b] = [a,b]$

$[a,b] \nabla [c,d] = [c < a ? -\infty : a, b < d ? +\infty : b]$

Widening: Exercise

$[1,2] \nabla [0,2] = [-\infty, 2]$

$[0,2] \nabla [1,2] = [0,2]$

$[1,5] \nabla [1,5] = [1,5]$

$[2,3] \nabla [2,4] = [2,+\infty]$

Definition:

$[a,b] \nabla \bot = [a,b]$ and $\bot \nabla [a,b] = [a,b]$

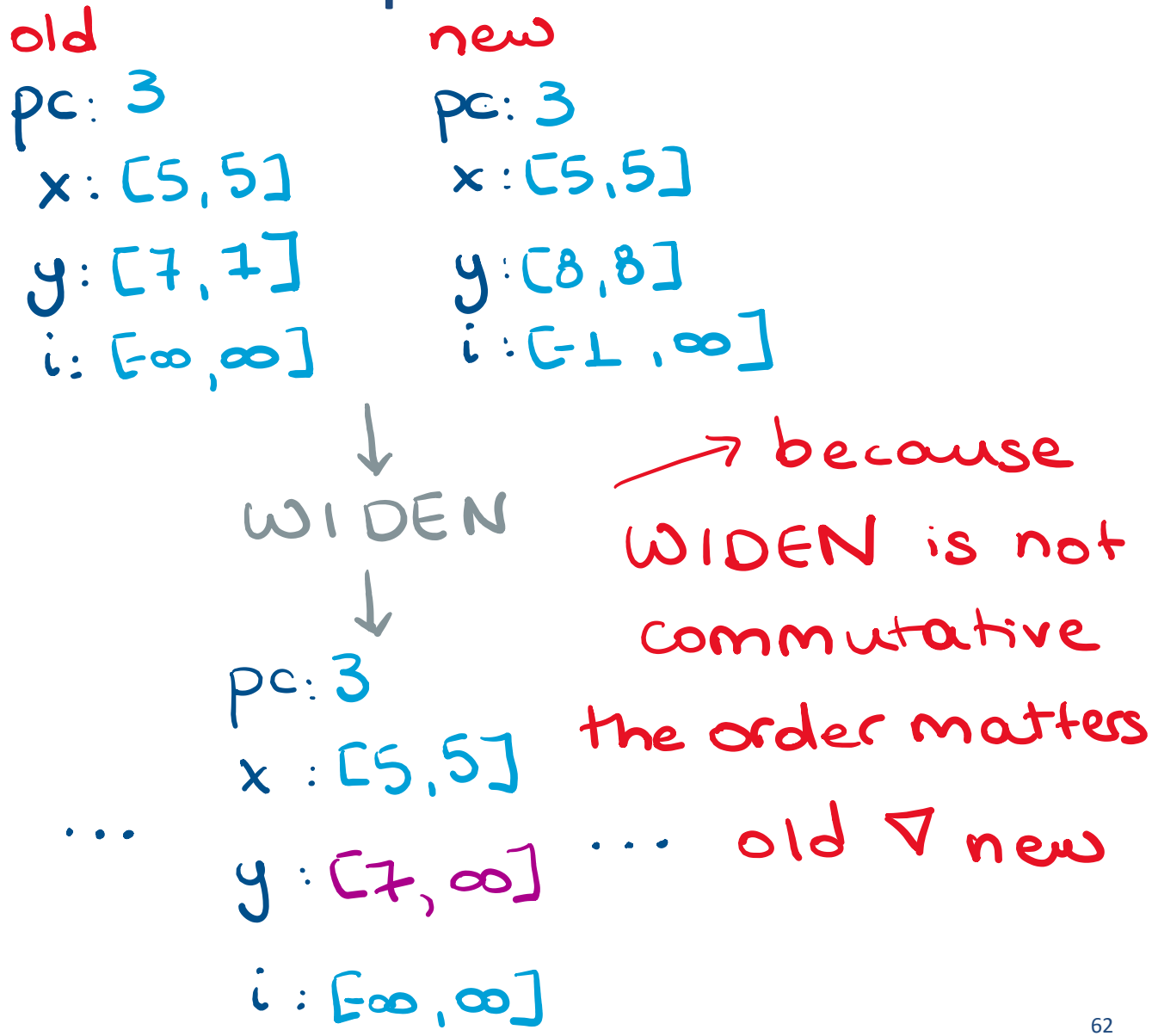
$[a,b] \nabla [c,d] = [c < a ? -\infty : a, b < d ? +\infty : b]$

Example – Step 3: Iterate to fixed point

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= y - x;
}
```



Example – Step 4: Check property

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= y - x;
}
```

pc: 7

... x : [5, 5] ...

y : [7, ∞]

i : [-∞, -1]

$P \models (0 \leq y - x)$ ✓

$P_{int} \quad (0 \leq y - x)$

Example – Step 4: Check property

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= y - x;
}
```

pc: 7

... x : [5, 5] ...

y : [7, ∞]

i : [-∞, -1]

$P \models (0 \leq y - x)$ ✓

$P_{int} \models (0 \leq y - x)$ ✓

Example – Step 4: Check property

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= y - x;
}
```

pc: 7

... x : [5, 5] ...

y : [7, ∞]

i : [-∞, -1]

$P \models (0 \leq y - x)$ ✓

$P_{int} \models (0 \leq y - x)$ ✓

The interval domain is precise enough to prove the property

Relational abstract domains

- The sign and interval domains are non-relational abstract domains
  - They do not relate different program variables
- Relational domains track relationships between variables and are more powerful
- For example, the assertion on the right cannot be proved with the interval domain

```
void Bar(int i) {
  1: int x = 0;
  2: int y = 0;

  3: if (i >= 0) {
  4:   x = x + 1;
  5:   y = y + 1;
  6:   i = i - 1;
  7:   goto 3;
     }

  8: assert x == y;
}
```

Relational domains: Examples

- The octagon domain tracks constraints of the form

$$\pm x \pm y \leq c$$

- The polyhedra domain tracks constraints of the form

$$c_1 x_1 + \cdots + c_n x_n \leq c$$

- The polyhedra domain is the most precise out of all the domains we have discussed, but it can be very expensive (exponential complexity)

Abstract interpretation: The good

- Automatically proves a program is free of errors (when sound)
  - Once the abstract transformers are known, we can analyze any program
- Based on a solid mathematical foundation
  - Well defined notions of soundness and approximation
- Has been widely used for many problems
  - Flight control software, device drivers, algorithm verification, code search, code synthesis, biological systems, …

Abstract interpretation: The bad

- Crafting the abstract transformers may be tricky
  - We must worry about soundness and precision
- The abstract transformers can be expensive and not scale to large programs
- False positives: It can be difficult to tell if a reported bug is due to over-approximation or if it is a real error

Abstract interpreter for C/C++ (actually LLVM)

- Clam: https://github.com/seahorn/clam

Abstract interpretation: Application

Perfectly Parallel Causal-Fairness Certification of Neural Networks

Joint work with

Caterina Urban, Valentin Wüstholz, and Fuyuan Zhang

Bias in machine learning

- Neural networks play an increasingly important role in our everyday decision making
  - Criminal justice, health care, social welfare, etc.

Bias in machine learning

- Neural networks play an increasingly important role in our everyday decision making
  - Criminal justice, health care, social welfare, etc.

- However, there are serious concerns about their fairness
  - Recidivism risk, health, image search, facial recognition, etc.

Bias in machine learning

- Neural networks play an increasingly important role in our everyday decision making
  - Criminal justice, health care, social welfare, etc.

- However, there are serious concerns about their fairness
  - Recidivism risk, health, image search, facial recognition, etc.

- Neural networks may reproduce, or even reinforce, bias that is directly or indirectly present in the training data

Fairness of neural networks

- It is critical to develop tools and techniques for certifying fairness of neural networks

Fairness of neural networks

- It is critical to develop tools and techniques for certifying fairness of neural networks

- We have designed a static-analysis framework for certifying causal fairness of feed-forward neural networks used for classification tasks of tabular data

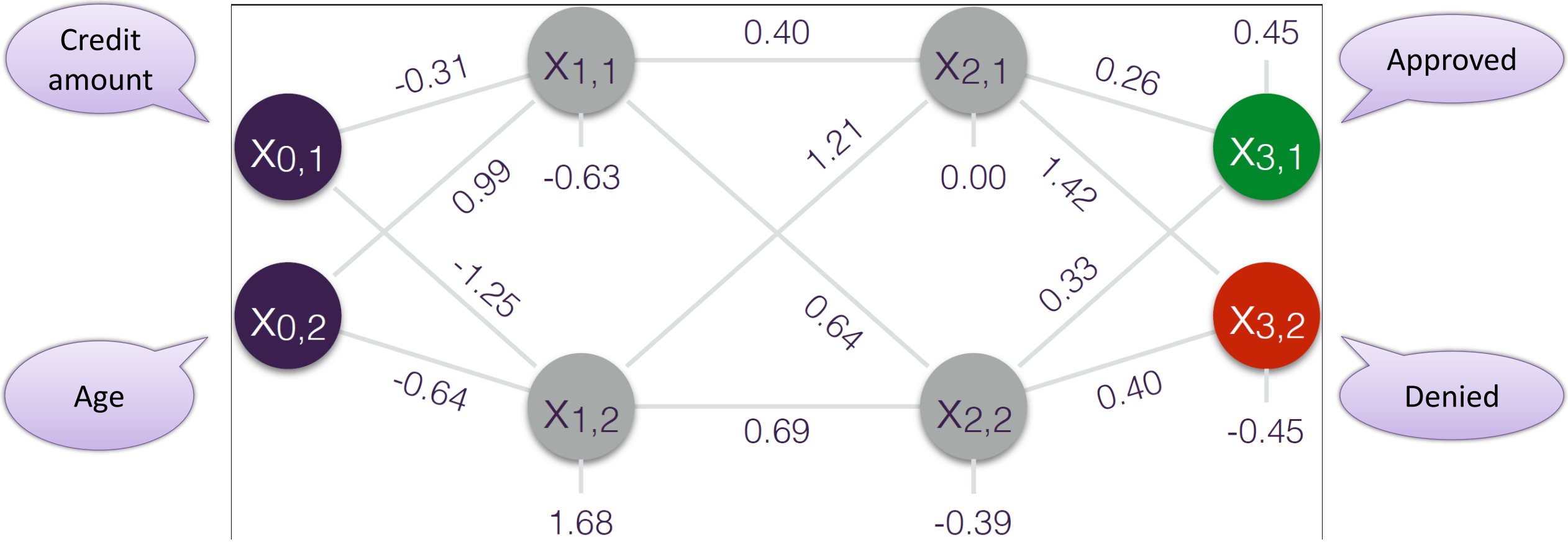
Fairness of neural networks

- It is critical to develop tools and techniques for certifying fairness of neural networks

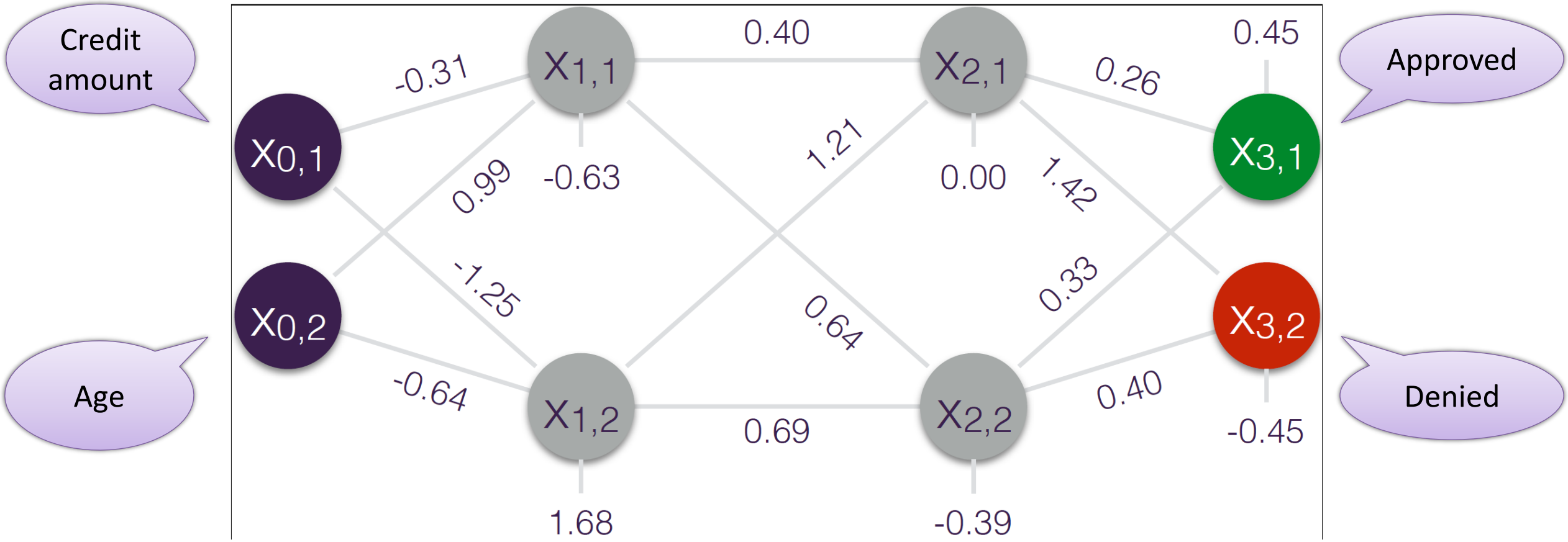
- We have designed a static-analysis framework for certifying causal fairness of feed-forward neural networks used for classification tasks of tabular data

  - Given input features that are sensitive to bias (e.g., race or gender), a neural network is causally fair if the output classification remains unaffected by different values of the sensitive features

Example


Credit amount
Age
Approved
Denied
X0,1
X0,2
X1,1
X1,2
X2,1
X2,2
X3,1
X3,2
-0.31
0.99
-0.63
-1.25
-0.64
1.68
0.40
1.21
0.64
0.69
0.00
-0.39
0.26
1.42
0.33
0.40
0.45
-0.45

Example



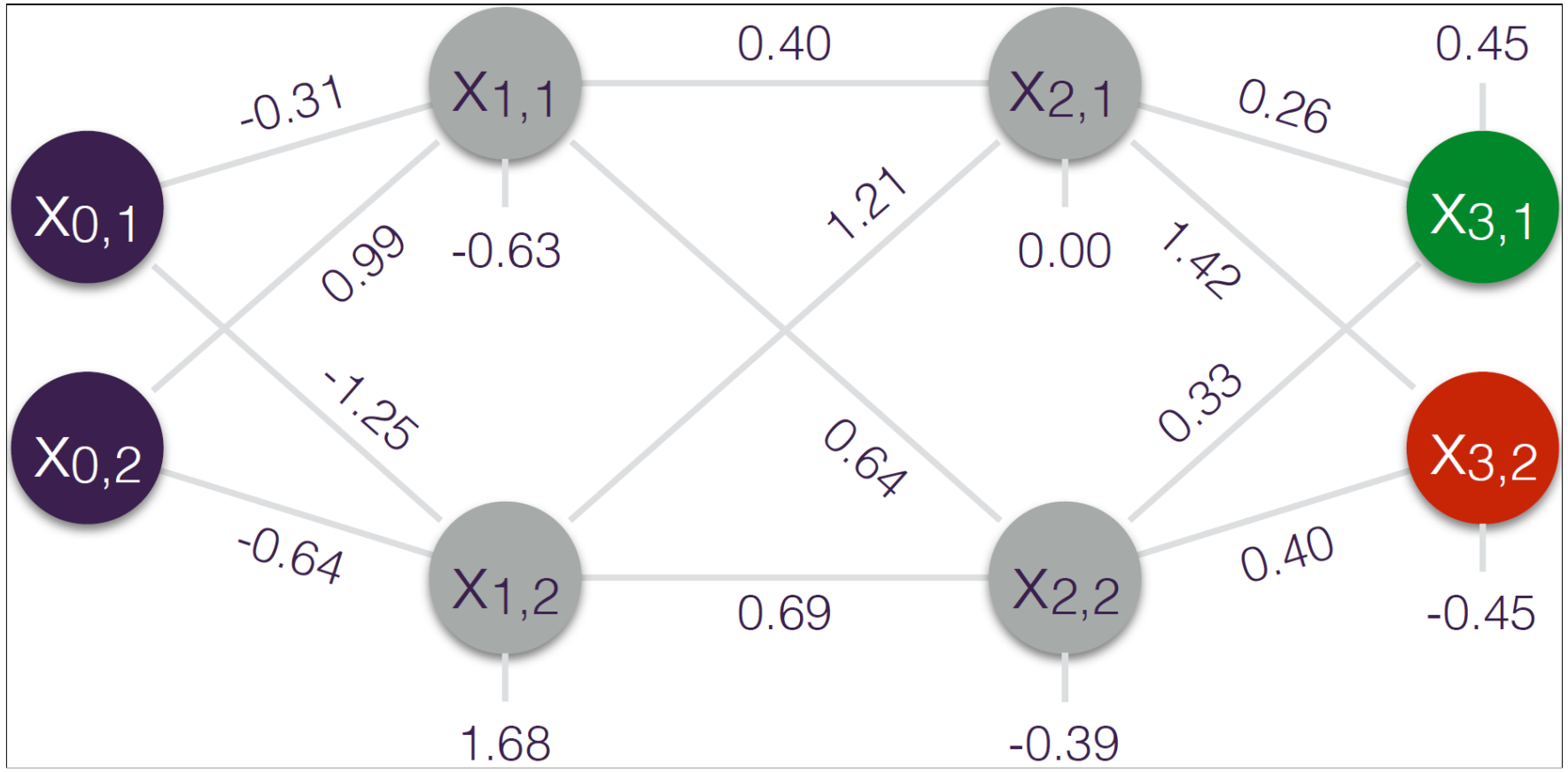
The network denies requests for large credit amounts from older people

Naïve approach

1. Analyze the neural network backwards, starting from each output

Naïve approach

1. Analyze the neural network backwards, starting from each output



Naïve approach

1. Analyze the neural network backwards, starting from each output
   - Credit is denied for $10'000 < x_{0,1}$ and $60 < x_{0,2}$
   - Credit is approved for $x_{0,1} \leq 10'000$ and $60 < x_{0,2}$ or for $x_{0,2} \leq 60$

Naïve approach

1. Analyze the neural network backwards, starting from each output
   - Credit is denied for $10'000 < x_{0,1}$ and $60 < x_{0,2}$
   - Credit is approved for $x_{0,1} \leq 10'000$ and $60 < x_{0,2}$ or for $x_{0,2} \leq 60$

2. Forget the value of the sensitive feature, i.e., project these regions over the remaining features

Naïve approach

1. Analyze the neural network backwards, starting from each output
   - Credit is denied for $10'000 < x_{0,1}$ and $60 < x_{0,2}$
   - Credit is approved for $x_{0,1} \leq 10'000$ and $60 < x_{0,2}$ or for $x_{0,2} \leq 60$

2. Forget the value of the sensitive feature, i.e., project these regions over the remaining features
   - Credit is denied for $10'000 < x_{0,1}$
   - Credit is approved for any $x_{0,1}$

Naïve approach

1. Analyze the neural network backwards, starting from each output
   - Credit is denied for $10'000 < x_{0,1}$ and $60 < x_{0,2}$
   - Credit is approved for $x_{0,1} \leq 10'000$ and $60 < x_{0,2}$ or for $x_{0,2} \leq 60$

2. Forget the value of the sensitive feature, i.e., project these regions over the remaining features
   - Credit is denied for $10'000 < x_{0,1}$
   - Credit is approved for any $x_{0,1}$

3. Intersect the projected regions, a non-empty intersection denotes bias w.r.t. the sensitive feature

Naïve approach

1. Analyze the neural network backwards, starting from each output
   - Credit is denied for $10'000 < x_{0,1}$ and $60 < x_{0,2}$
   - Credit is approved for $x_{0,1} \leq 10'000$ and $60 < x_{0,2}$ or for $x_{0,2} \leq 60$

2. Forget the value of the sensitive feature, i.e., project these regions over the remaining features
   - Credit is denied for $10'000 < x_{0,1}$
   - Credit is approved for any $x_{0,1}$

3. Intersect the projected regions, a non-empty intersection denotes bias w.r.t. the sensitive feature
   - Bias is found for $10'000 < x_{0,1}$